

A Core Calculus for Signature-First Reactive Design

Persistent Declarations, Nominal Concepts, and One Temporal Primitive

ZHU ZHEHAO

Formal development KCN-judu/BDL_FV (Lean 4.33.1, through Phase 18); paper revision of 2026-09-20

Abstract

Interactive physical products are designed by people who state *relationships* before they can define them: a lamp’s brightness follows its tilt; the room’s temperature is read once a second while the tilt is read fifty times a second; the light is the product’s only physical effect. We present λ_{BDL} , the core calculus of the Behavior Design Language, in which such a design is a finite environment of *persistent declarations* — each a stable identity, a frozen expected type with a growable set of public commitments, and an optional realization — over a simply typed term language extended with nominal *concept* types bound write-once to data representations, a *construction grant* derived from the declaration’s own signature, physical *dimensions* carried by operator types, ordinary data (options, lists, products) with one list recursor, and exactly one temporal primitive: read a *clock domain* at its last activation strictly before now. We give the calculus a tick-indexed big-step semantics and prove it deterministic and total on *causal* designs by a tick-indexed logical relation with a lexicographic induction on (tick, instantaneous rank, derivation); the single-domain semantics is the diagonal of the multi-domain one, memory is transport at the own domain, and no scheduler order is observable. We prove that refining a declaration — adding a commitment, supplying a body, strengthening a realized interface with re-verification — preserves every client’s typing unconditionally and every client’s discharged commitment provided the validation layer’s evidence is monotone in the environment, and we show by a mechanized counterexample that the monotonicity condition is necessary. Semantic identity has no runtime residue (erasure is sound), cannot be manufactured except where a signature announces it (construction is granted, and provenance is preserved through memory and transport), and is orthogonal to dimension. Physical effect passes through explicit drive edges with a single driver per output, which makes the physical output a function of the tick. Behavior components are derived from renaming and realization alone, and flattening a composition yields an ordinary design checked by the unchanged judgments. Every definition and theorem is mechanized in Lean 4 with no sorry and no classical choice; every rejected alternative is a mechanized counterexample; every trace is computed by an interpreter proved sound for the relation.

Keywords: reactive semantics, clock domains, nominal types, refinement, logical relations, mechanized metatheory, Lean 4, synchronous languages, units of measure

1 Introduction

A behavior designer working on an interactive physical product does something that a programmer’s language makes awkward: they *declare a relationship they cannot yet define*. “Brightness follows tilt” is a complete design statement long before anyone knows the formula; it has a type — the concept `Tilt` to the concept `Brightness` — and other parts of the design depend on it at that type. Later the relationship acquires a formula, a promise (“monotone”), a timing domain, a physical output; none of these later steps should disturb what already depended on it. The formula must be unable to produce anything but a `Brightness`, even though a brightness and an opacity are both represented by a dimensionless number and a tilt and a motor angle are both angles. Values must be remembered across time with an explicit first value, and read across timing domains without making the scheduler visible. Physical effect must happen exactly once per output, explicitly.

This paper presents the core calculus that gives these requirements a semantics and proves that they compose: λ_{BDL} , the kernel of the Behavior Design Language (BDL). The calculus is deliberately small. Its term language is

a simply typed λ -calculus with registered first-order operators; what makes it a *design* calculus is not its terms but the environments they are typed and evaluated against, and the two disciplines those environments impose — a *refinement order* on declarations, and a *grant* on the construction of nominal values.

1.1 What the calculus adds to the simply typed λ -calculus

- **Persistent declarations** (§4). A design is an environment Δ of declarations $\langle id, \langle \tau, C \rangle, realization \rangle$ with a stable identity, a frozen expected type, a growable list of public commitments, and an optional body. Terms refer to declarations by identity, and the typing judgment reads Δ through exactly one projection, the *type view*. A refinement order on declarations and environments captures the operations that a client may survive; everything else is an *edit* about which nothing is promised. We prove that refinement preserves every client's typing with no hypothesis, and every client's discharged commitment under a Kripke-style monotonicity condition on the validation layer's evidence — a condition we show necessary (Theorems 4–5, Proposition 6).
- **Nominal concepts with granted construction** (§5). A concept is a nominal type sem s ; a concept environment Θ binds it, write-once, to a representation type that is data and mentions no concept. Observing a representation (rep) is always permitted; constructing a value (mk s) requires a grant, and a declaration's body is granted exactly the concepts in result position of its own signature. Under that discipline a hidden crossing between concepts is untypable, erasure to the representation is sound, and no tag is created by evaluation, memory, or transport (Theorems 8, 9 and 14).
- **Dimensions in operator types** (§5.3). A physical quantity has type $q\ d$ for an exponent vector d ; the algebra lives entirely in the registered operators' types and no typing rule mentions it. The dimensionless baseline is the erasure of this typing, exactly as the numeric baseline is the erasure of nominal typing.
- **One temporal primitive** (§6–7). `sync  $c$  init  $e$` reads e in clock domain c at c 's last activation strictly before the current tick, or *init* if there is none; delay is sync at the own domain. The semantics is tick-indexed big-step evaluation against an input stream and a schedule. It is deterministic unconditionally and total on *causal* designs — those whose instantaneous dependency graph is acyclic — by a logical relation indexed by the tick with a lexicographic induction on tick, rank and derivation (Theorems 10–13, 18–19). The delayed type must be data and delay may appear only at top level; both restrictions are forced by the totality proof rather than chosen. Clock domains are nominal identities, not rates; the *strictly before* rule makes the semantics independent of any order between simultaneously active domains, and the alternative is mechanically shown to expose the scheduler (Theorem 20).
- **Data with one recursor** (§8). Options, lists and products are ordinary data; fold is the one term former that applies a function value during evaluation. Its evaluation rule unrolls syntactically through the environment so that the evaluation relation remains an ordinary inductive relation. Polymorphism is rank-1 by definitional families instantiated by one-way matching against closed types; no type variable enters the kernel. The lossless cross-domain window buffer — the one construction a designer would expect to be primitive — is five declarations over delay, sync and list operators, with a correctness theorem for every schedule (Theorem 21).
- **Explicit outputs** (§9). A declaration computes a value; hardware moves only through a drive edge to a nominally identified output with a single driver. With one driver the physical output is a partial function of the tick; with a hidden arbitration policy three different outputs arise from one design (Theorem 23).
- **Composition by renaming** (§10). Behavior components, fresh instantiation, bindings and flattening are derived from realization steps and an equivariance theorem for every judgment; the flattened system is an ordinary design accepted by the unchanged checkers (Theorems 24–26).

1.2 Mechanization and claims

Every definition, theorem and counterexample in this paper is a Lean 4 declaration in the development `KCN-judu/BDL_FV` [1]. The development builds with no `sorry`; the axioms used are propositional extensionality and quotient soundness (the latter only through function extensionality), and classical choice is absent. Negative results are theorems whose content is a rejection; every trace is computed by an interpreter proved sound for the evaluation relation, inside the proof checker. The paper cites the Lean name of each result at the point it is stated, and Appendix A indexes them by section.

Two conventions bound the claims. *Minimal* always means minimal among the alternatives that were formalized and refuted, never a minimality theorem. Where a theorem is proved for a fragment — the modular-semantics result holds for single-domain wiring designs with direct bindings — the restriction is part of the statement.

1.3 What this paper is not

It is not a paper about the surface language, the authoring environment or the production toolchain that implements the calculus; those are described in the BDL monograph, of which this paper is the formal core. It is not a paper about hardware validation or deployment, which live above the kernel as decidable relations that never enter typing or evaluation. And it is not a new type-theoretic mechanism: each ingredient is a known shape — the interface/implementation separation of module signatures [2], [3], an abstract type with a private constructor [4], a Kripke-style stability condition, the *pre* of Lustre [5] confined to nodes, a step-indexed logical relation [6], [7]. What is specific is where the ingredients come from — the grant from the signature the designer already wrote, the domain from an authored identity rather than an inferred clock, the buffer from two reads of a log — and that their combination has been proved to compose.

2 Overview: a lamp in λ_{BDL}

We fix a small product to make the calculus concrete: a lamp whose brightness follows its tilt, dims smoothly, and warms its base according to the room temperature. The design begins with concepts and signatures and nothing else.

Concepts. *Tilt*, *Brightness*, *RoomTemp* and *MotorAngle* are semantic identities $s \in \text{SemanticId}$. Each is a nominal type $\text{sem } s$. The concept environment Θ binds *Tilt* and *MotorAngle* to q Angle and *Brightness* to q 0 ; the two angles remain distinct types.

Declarations before definitions. The relationship $\text{dimByTilt} : \text{Tilt} \rightarrow \text{Brightness}$ is a declaration $\langle d_1, \langle \text{sem Tilt} \rightarrow \text{sem Brightness}, [] \rangle, \text{none} \rangle$: an identity, an interface, and no body. $\text{tilt} : \text{Tilt}$ is a declaration with no body that will never receive one — an *input*, whose value the environment supplies at every tick. $\text{light} : \text{Brightness}$ is realized as $\text{app } (\text{declRef } d_1) (\text{declRef tilt})$ and is well typed now, against d_1 's interface, whether or not d_1 ever acquires a formula.

Refinement. Giving dimByTilt the body $\lambda x : \text{sem Tilt} . \text{mk Brightness } (\text{clamp } (\text{rep } x / 90^\circ) 0 1)$ — where 90° is surface notation for a dimensioned literal $\text{lit}_{\text{Angle}}$ and clamp for a library term over lt and ite — is a refinement step. The body is typed under the grant $\{\text{Brightness}\}$ — the concepts in result position of $\text{Tilt} \rightarrow \text{Brightness}$ — so mk Brightness is permitted and mk MotorAngle would not be. light 's typing is untouched (Theorem 4). Adding the commitment *monotone* to dimByTilt later is a refinement; changing its expected type is an edit.

Memory. A smooth dimmer remembers its previous output: $\text{smooth} : \text{Brightness}$ realized as a mix of light and delay *init* (declRef smooth). The self-reference is a structural cycle every path of which passes through a delayed operand; the design is *causal* and has a value at every tick (Theorem 13). The initial value is part of the syntax: without it the first tick is undefined or nondeterministic, and both failure modes are mechanized.

Clock domains. tilt and light live in a domain *fast*; roomTemp and heat in a domain *slow*. Domains are nominal: a clone of *fast* with the same schedule is a different domain, and a direct wire between them is rejected by the domain judgment. heat may read the tilt only through $\text{sync fast init } (\text{declRef light})$, whose value is light at the last *fast* activation strictly before the current *slow* tick. Typing is blind to domains and unchanged.

Output. The physical light is an output identity o accepting sem Brightness in domain *fast*; the drive edge $\beta \text{ smooth} = \text{some } o$ is well formed because the types are equal and the domains agree, and it is the only driver of o . Nothing else in the design has a physical effect.

Sections 3–10 give each of these steps its rules and theorems.

3 The calculus

3.1 Syntax

Figure 1 gives the syntax. Types are those of a simply typed calculus with booleans, counts and arrows, extended by nominal concept types $\text{sem } s$ over an identity s , physical quantities $q \ d$ over a dimension d , and the data formers opt , list and $\times$. A dimension is an exponent vector over a fixed finite set of base dimensions (length, time, angle, mass, temperature in the development); dimensions form an abelian group under pointwise addition, which is the only structure the calculus uses.

$$\begin{aligned}
& s \in \text{SemanticId} \quad d \in \text{Dim} \quad c \in \text{ClockId} \quad \delta \in \text{DeclId} \quad o \in \text{OutputId} \\
& \tau, \sigma ::= \text{bool} \mid \text{nat} \mid \tau \rightarrow \sigma \mid \text{sem } s \mid q \ d \mid \text{opt } \tau \mid \text{list } \tau \mid \tau \times \sigma \\
& e ::= x \mid \text{true} \mid \text{false} \mid n \mid \lambda x : \tau. e \mid e \ e \mid \text{declRef } \delta \mid \text{rep } e \mid \text{mk } s \ e \mid p \\
& \quad \mid \text{delay } e \ e \mid \text{sync } c \ e \ e \mid \text{fold } e \ e \ e \\
& p ::= \text{lit}_d \ n \mid \text{add}_d \mid \text{sub}_d \mid \text{mul}_{d_1 d_2} \mid \text{div}_{d_1 d_2} \mid \text{lt}_d \mid \text{eq}_\tau^h \mid \neg \mid \wedge \mid \vee \mid \text{ite}_\tau \\
& \quad \mid \text{none}_\tau \mid \text{some}_\tau \mid \text{isSome}_\tau \mid \text{getD}_\tau \mid \text{nil}_\tau \mid \text{cons}_\tau \mid \text{length}_\tau \mid \text{take}_\tau \mid \text{drop}_\tau \mid \text{reverse}_\tau \mid \text{head}_\tau \\
& \quad \mid \text{toList}_\tau \mid \text{pair}_{\tau\sigma} \mid \text{fst}_{\tau\sigma} \mid \text{snd}_{\tau\sigma}
\end{aligned}$$

Figure 1. Syntax of λ_{BDL} . Variables are de Bruijn indices in the development; the paper writes names. h in eq_τ^h is a proof that τ is a data type.

Terms are those of the λ -calculus plus five design-specific forms. $\text{declRef } \delta$ refers to a declaration by identity; nothing about the declaration's interface or body is in the syntax. $\text{rep } e$ observes the representation of a concept value and $\text{mk } s \ e$ constructs one. $\text{delay } i \ e$ is the value of e at the previous activation of the current domain, i before any; $\text{sync } c \ i \ e$ is the value of e in domain c at c 's last activation strictly before now, i if none. $\text{fold } f \ z \ l$ is the list recursor, $\text{fold } f \ z \ [x_1, \dots, x_n] = f \ x_1 \ (\dots (f \ x_n \ z))$. Registered operators p are first-order constants with types; they never apply a closure.

Two predicates on types recur. A type is **data**, τ . Data, when no arrow occurs in it; a type is **concept-free**, τ . SemFree , when no sem occurs in it. Both are decidable by structural recursion, and $(\tau \times \sigma)$. $\text{Data} \Leftrightarrow \tau$. $\text{Data} \wedge \sigma$. $\text{Data}, (\text{list } \tau)$. $\text{Data} \Leftrightarrow \tau$. Data hold definitionally $(\text{Ty.prod_data}, \text{Ty.list_data})$.

3.2 Environments

A term is typed and evaluated against several environments, each read through a stated projection and nothing else. This discipline — *which environment a judgment may see* — is what the metatheory rests on, and each theorem's hypotheses name the environments it depends on.

- A **declaration** is a triple

$$\text{DesignDecl} = \langle id : \text{DeclId}, interface : \langle expectedType : \text{Ty}, commitments : \text{PropertyId}^* \rangle, realization : \text{Option Expr} \rangle.$$

A **design** is a declaration environment $\Delta : \text{DeclId} \rightarrow \text{Option DesignDecl}$. Its *type view* $\Delta^{\text{ty}}(\delta) = (\Delta \ \delta)$. $\text{map}(\cdot . interface . expectedType)$ is all that typing sees; its *realization view* $\Delta^{\text{real}}(\delta) = (\Delta \ \delta)$. $\text{bind}(\cdot . realization)$ is all that evaluation sees. An **unresolved** declaration is one whose realization is none; nothing else distinguishes it.

- A **concept environment** $\Theta : \text{SemanticId} \rightarrow \text{Option Ty}$ binds each concept to a representation. It is well formed, Θ . WF , when every bound representation is concept-free and data: $\Theta \ s = \text{some } R \Rightarrow R$. $\text{SemFree} \wedge R$. Data .
- A **grant** $G : \text{SemanticId} \rightarrow \text{Prop}$ says which concepts a term may construct. Grant.none permits nothing; $\text{Grant.of } \tau$ permits the concepts in result position of τ , $\text{grant}(\text{sem } s) = [s]$, $\text{grant}(\tau \rightarrow \sigma) = \text{grant}(\sigma)$, $\text{grant}(_) = []$.
- A **clock environment** $K : \text{DeclId} \rightarrow \text{Option ClockId}$ assigns each declaration a domain; none is a domain-agnostic pure mapping. A **schedule** $S : \text{ClockId} \rightarrow \mathbb{N} \rightarrow \text{Bool}$ says at which global ticks each domain activates. An **input** $I : \text{DeclId} \rightarrow \mathbb{N} \rightarrow \text{Value}$ supplies a value for every unresolved declaration at every tick.

- An **output environment** $\Omega : \text{OutputId} \rightarrow \text{Option } \langle \text{accepts} : \text{Ty}, \text{clock} : \text{ClockId} \rangle$ and the **drive edges** $\beta : \text{DeclId} \rightarrow \text{Option OutputId}$ are introduced in §9.

Typing sees Θ , Δ^{ty} and G . Evaluation sees Δ^{real} , I and (in several domains) S . The domain judgment sees K . Outputs see Ω , K , Δ^{ty} and β . Commitments and evidence are seen by the satisfaction relation of §4 and by nothing else.

3.3 Typing

The typing judgment $\Theta; \Delta; G; \Gamma \vdash e : \tau$ is given in Figure 2. Rules T-Var, T-Bool, T-Nat, T-Lam and T-App are those of the simply typed λ -calculus. T-Ref is the only rule that reads Δ , and it reads the type view. T-Rep and T-Mk read Θ through the binding $\Theta s = \text{some } R$; T-Mk additionally requires the grant. T-Prim assigns each registered operator its type; the dimension algebra is entirely in that table (Figure 3), so an application of $\text{mul}_{d_1 d_2}$ is checked by T-App like any other. T-Delay and T-Sync require the type to be data and the context to be empty; T-Fold types the recursor.

$$\begin{array}{c}
\frac{\Gamma(x) = \tau}{\Theta; \Delta; G; \Gamma \vdash x : \tau} \text{ (T-Var)} \quad \frac{}{\Theta; \Delta; G; \Gamma \vdash b : \text{bool}} \text{ (T-Bool)} \quad \frac{}{\Theta; \Delta; G; \Gamma \vdash n : \text{nat}} \text{ (T-Nat)} \\
\\
\frac{\Theta; \Delta; G; \Gamma, x : \tau \vdash e : \sigma}{\Theta; \Delta; G; \Gamma \vdash \lambda x : \tau. e : \tau \rightarrow \sigma} \text{ (T-Lam)} \quad \frac{\Theta; \Delta; G; \Gamma \vdash f : \tau \rightarrow \sigma \quad \Theta; \Delta; G; \Gamma \vdash a : \tau}{\Theta; \Delta; G; \Gamma \vdash f a : \sigma} \text{ (T-App)} \\
\\
\frac{\Delta^{\text{ty}}(\delta) = \text{some } \tau}{\Theta; \Delta; G; \Gamma \vdash \text{declRef } \delta : \tau} \text{ (T-Ref)} \quad \frac{}{\Theta; \Delta; G; \Gamma \vdash p : \text{ty}(p)} \text{ (T-Prim)} \\
\\
\frac{\Theta s = \text{some } R \quad \Theta; \Delta; G; \Gamma \vdash e : \text{sem } s}{\Theta; \Delta; G; \Gamma \vdash \text{rep } e : R} \text{ (T-Rep)} \quad \frac{G s \quad \Theta s = \text{some } R \quad \Theta; \Delta; G; \Gamma \vdash e : R}{\Theta; \Delta; G; \Gamma \vdash \text{mk } s e : \text{sem } s} \text{ (T-Mk)} \\
\\
\frac{\tau. \text{Data} \quad \Theta; \Delta; G; [] \vdash i : \tau \quad \Theta; \Delta; G; [] \vdash e : \tau}{\Theta; \Delta; G; [] \vdash \text{delay } i e : \tau} \text{ (T-Delay)} \\
\\
\frac{\tau. \text{Data} \quad \Theta; \Delta; G; [] \vdash i : \tau \quad \Theta; \Delta; G; [] \vdash e : \tau}{\Theta; \Delta; G; [] \vdash \text{sync } c i e : \tau} \text{ (T-Sync)} \\
\\
\frac{\Theta; \Delta; G; \Gamma \vdash f : \tau \rightarrow \sigma \rightarrow \sigma \quad \Theta; \Delta; G; \Gamma \vdash z : \sigma \quad \Theta; \Delta; G; \Gamma \vdash l : \text{list } \tau}{\Theta; \Delta; G; \Gamma \vdash \text{fold } f z l : \sigma} \text{ (T-Fold)}
\end{array}$$

Figure 2. Typing (*HasType*). T-Ref is the only rule reading Δ ; T-Rep and T-Mk the only rules reading Θ ; T-Mk the only rule reading G .

operator	type	operator	type
$\text{lit}_d n$	$\text{q } d$	eq_τ^h	$\tau \rightarrow \tau \rightarrow \text{bool}$
$\text{add}_d, \text{sub}_d$	$\text{q } d \rightarrow \text{q } d \rightarrow \text{q } d$	ite_τ	$\text{bool} \rightarrow \tau \rightarrow \tau \rightarrow \tau$
$\text{mul}_{d_1 d_2}$	$\text{q } d_1 \rightarrow \text{q } d_2 \rightarrow \text{q } (d_1 + d_2)$	some_τ	$\tau \rightarrow \text{opt } \tau$
$\text{div}_{d_1 d_2}$	$\text{q } d_1 \rightarrow \text{q } d_2 \rightarrow \text{q } (d_1 - d_2)$	getD_τ	$\text{opt } \tau \rightarrow \tau \rightarrow \tau$
lt_d	$\text{q } d \rightarrow \text{q } d \rightarrow \text{bool}$	toList_τ	$\text{opt } \tau \rightarrow \text{list } \tau$
length_τ	$\text{list } \tau \rightarrow \text{q } 0$	cons_τ	$\tau \rightarrow \text{list } \tau \rightarrow \text{list } \tau$
head_τ	$\text{list } \tau \rightarrow \text{opt } \tau$	$\text{take}_\tau, \text{drop}_\tau$	$\text{q } 0 \rightarrow \text{list } \tau \rightarrow \text{list } \tau$
$\text{fst}_{\tau\sigma}$	$\tau \times \sigma \rightarrow \tau$	$\text{pair}_{\tau\sigma}$	$\tau \rightarrow \sigma \rightarrow \tau \times \sigma$

Figure 3. Types of the registered operators (*Prim.ty*), abridged. Dimension algebra lives here and nowhere else. eq is available at every data type and lt at quantities only.

Three features of Figure 2 carry the rest of the paper.

The typing boundary. Typing depends on the type view of declarations and the representation view of concepts and on nothing else — not on realizations, commitments, evidence, clocks or drive edges. The client-stability theorem of §4 is a one-line consequence, and its necessity is a one-line counterexample.

The construction boundary. Client code is typed under `Grant.none`; a declaration’s realization is typed under `Grant.of` its own expected type (§4.1). A value of sem s is therefore constructed only inside a declaration whose signature announces sem s . This is the whole of the semantic-isolation mechanism, and §5 shows what each weaker alternative admits.

The temporal boundary. `delay` and `sync` are typed only in the empty context and only at data types. Both restrictions were forced by the totality proof of §6, not chosen: a delayed closure would have to be transported across ticks, and a delay under a binder would re-evaluate its operand at the previous tick in an environment created at the current one. Temporal state therefore belongs to declarations, and mappings are pointwise – the arrangement of `pre` in Lustre, where it lives in nodes rather than in functions [5].

3.4 Inference, uniqueness and monotonicity

Inference is syntax-directed. A function `infer` $\Theta \Delta G \Gamma e : \text{Option Ty}$ follows the rules of Figure 2 and needs only decidability of G , of type equality and of τ . `Data`.

Theorem 1 (Inference; `infer_sound`, `infer_complete`, `HasType.unique`). `infer` $\Theta \Delta G \Gamma e = \text{some } \tau$ iff $\Theta; \Delta; G; \Gamma \vdash e : \tau$; hence typing is decidable and every term has at most one type.

Uniqueness matters beyond decidability: it is why the surface language’s polymorphism can be *matching* rather than unification (§8.4), and why a nominal mismatch is reported as “Brightness and Opacity are different concepts” and never as a unification residue.

Theorem 2 (Monotonicity; `HasType.mono_env`, `HasType.mono_concept`, `HasType.mono_grant`). Typing is monotone in each of its three environments: if $\Theta; \Delta; G; \Gamma \vdash e : \tau$, then the same holds in any Δ' with `EnvRefines` $\Delta \Delta'$ (§4.2), any Θ' that binds at least what Θ binds, and any $G' \supseteq G$.

Weakening holds for the delay-free fragment by appending to the context (`HasType.weaken_append`); a stateful term cannot be moved under a binder at all, so no stronger weakening is needed.

4 Declarations, interfaces and refinement

The foundational object is the declaration, and the foundational question is which changes to a declaration its clients survive. This section fixes the answer: a *refinement order*, the three steps that generate it, and two preservation theorems with deliberately different hypotheses.

4.1 Interfaces, evidence and satisfaction

An interface $S = \langle \tau, C \rangle$ is an expected type and a list of commitments – atomic labels such as `total`, `monotone`, bounded that a client may rely on. Interfaces are ordered by monotone refinement:

$$S \sqsubseteq S' := S.\tau = S'.\tau \wedge S.C \subseteq S'.C,$$

a decidable preorder, frozen on the type and growing on commitments (`InterfaceRefines`). Nothing else is an interface refinement.

What discharges a commitment is not the kernel’s business; it is the validation layer’s. The kernel abstracts it as an **evidence** relation $ev : \text{DeclEnv} \rightarrow \text{Expr} \rightarrow \text{PropertyId} \rightarrow \text{Prop}$. Evidence takes the environment because compositional discharge needs it – “ A is monotone because B is committed to be monotone” consults B ’s interface. A body e **satisfies** S in Θ, Δ, Γ when it has the expected type under the grant of that type and every commitment is discharged:

$$\text{Satisfies } ev \Theta \Delta \Gamma e S := \Theta; \Delta; \text{Grant.of}(S.\tau); \Gamma \vdash e : S.\tau \wedge \forall p \in S.C. \text{ } ev \Delta e p.$$

A declaration is well formed when its body, if any, satisfies its interface; a design is **globally well formed**, `GlobalWF` $ev \Theta \Delta$, when every stored declaration sits under its own identity and is well formed in Δ at top level. An unresolved declaration is always well formed.

The refinement order is complete for abstract evidence: $S \sqsubseteq S'$ iff every realization of S' in every environment under every evidence relation realizes S (`InterfaceRefines_iff_semantic`). The proof of the converse instantiates evidence at “the property is in S' ’s list” and the body at a reference to a single declaration.

4.2 The refinement order and the lifecycle

A declaration takes a refinement step in one of three ways (`DeclRefines`), each preserving the identity by construction and each checked against the current environment Δ :

$$\frac{S \sqsubseteq S'}{\langle \delta, S, \text{none} \rangle \rightsquigarrow \langle \delta, S', \text{none} \rangle} \quad \frac{\text{Satisfies } ev \ \Theta \ \Delta \ \Gamma \ e \ S}{\langle \delta, S, \text{none} \rangle \rightsquigarrow \langle \delta, S, \text{some } e \rangle} \quad \frac{S \sqsubseteq S' \quad \text{Satisfies } ev \ \Theta \ \Delta \ \Gamma \ e \ S'}{\langle \delta, S, \text{some } e \rangle \rightsquigarrow \langle \delta, S', \text{some } e \rangle}$$

An unresolved declaration may have its interface refined; an unresolved declaration may be realized by a satisfying body; a realized declaration may have its interface strengthened provided the body is *re-verified* against the new interface. Strengthening without re-verification breaks well-formedness, and the counterexample is mechanized (`naive_breaks_wellformedness`).

Separately from the steps there is a purely structural order with no satisfaction condition: `DeclLeq` $h \ h'$ requires the same identity, $h.S \sqsubseteq h'.S$, and a write-once realization ($h.\text{realization} = \text{some } e \Rightarrow h'.\text{realization} = \text{some } e$); `EnvRefines` $\Delta \ \Delta'$ lifts it pointwise and permits new declarations. Storing a refined declaration back under its identity is an environment refinement — $\Delta \ h.\text{id} = \text{some } h \wedge \text{DeclLeq } h \ h' \Rightarrow \text{EnvRefines } \Delta \ (\Delta[h'])$ (`EnvRefines_update`) — and this is the one place identity does any work: it makes the update land on the slot every reference resolves to, which is what a name does in any environment semantics.

Theorem 3 (The lifecycle is the structural order; `DeclRefinesStar_iff`). The reflexive–transitive closure of the three steps, all side conditions checked in Δ , relates h to h' iff `DeclLeq` $h \ h'$ and h' is well formed in Δ .

4.3 Client stability

Can a declaration be refined or realized without editing its clients, and without invalidating what was established about them? The answer has two halves.

Theorem 4 (Client stability, typing; `local_refinement_preserves_global_typing`). If $\Delta \ B.\text{id} = \text{some } B$ and `DeclLeq` $B \ B'$, then every judgment $\Theta; \Delta; G; \Gamma \vdash e : \tau$ holds in $\Delta[B']$.

The proof is one line: typing reads Δ through the type view, and the type view is invariant under `DeclLeq`. The theorem should be read as such — its content is that letting clients see interfaces and never bodies is *sufficient* for client stability. It is also necessary: change B ’s expected type while keeping its identity and every client breaks; that is why the type is frozen in $\sqsubseteq$.

The commitment half needs more.

Definition (Monotone evidence). ev is **monotone** when `EnvRefines` $\Delta_1 \ \Delta_2 \wedge ev \ \Delta_1 \ e \ p \Rightarrow ev \ \Delta_2 \ e \ p$. Evidence that ignores the environment is monotone; evidence that consults only the *presence* of commitments and realizations is monotone; evidence that consults their *absence* is not.

Theorem 5 (Client stability, commitments; `local_refinement_preserves_global_wf`, `local_lifecycle_preserves_global_wf`). If ev is monotone, `GlobalWF` $ev \ \Theta \ \Delta$, $\Delta \ B.\text{id} = \text{some } B$ and $B \rightsquigarrow B'$ with side conditions checked in Δ , then `GlobalWF` $ev \ \Theta \ (\Delta[B'])$. The same holds for a whole lifecycle $B \rightsquigarrow^* B'$ checked against the original Δ .

Proposition 6 (Monotonicity is necessary; `badEv_not_mono`). There is an evidence relation ev_{bad} , a globally well formed two-declaration design, and a valid realization step of one declaration after which the design is not globally well formed; consequently ev_{bad} is not monotone.

The relation ev_{bad} discharges “ A is total” whenever the declaration A reads is still unresolved — evidence from absence. Realizing that declaration is a perfectly valid step, and it destroys the discharge. The monotonicity hypothesis was not part of the original design; it appeared when Theorem 5 was attacked, and it is a Kripke-style

stability condition on the validation layer: any discharge mechanism meant to survive refinement must be positive in the environment.

Binding a representation to a previously unbound concept is likewise a refinement: typing, satisfaction and global well-formedness are monotone in Θ (`GlobalWF.of_conceptRefines`). Rebinding a concept to a different representation is an edit that breaks existing realizations (`representation_change_is_edit_not_refinement`).

4.4 Refinement versus edit

Theorems 4–5 cover refinement only. Table 1 classifies the operations a tool offers on a declaration B read by a client A ; each row is witnessed by a mechanized example on a two-declaration design.

operation on B	kind	effect on A
add a public commitment	refinement	typing and commitments preserved
realize	refinement	preserved; A now unfolds to a closed program
strengthen a realized interface	refinement, with re-verification	preserved
change the expected type, keep identity	edit	typing broken
drop a commitment	edit	typing silent; A 's commitment broken
replace B by a new identity	edit	dangling reference
detach or replace the realization	edit	evidence that consulted the body is void
assign or change a clock domain (§7)	edit	domain judgment on clients broken
retarget an output binding (§9)	edit	completeness or single-driver may break

Table 1. *Refinement versus edit.*

Two rows are instructive. Dropping a commitment changes no type, so the type checker is silent, yet A 's own commitment was discharged through B 's and is now unsupported: commitments are part of the interface in the same load-bearing sense as the expected type. Detaching a realization is an edit for the same reason. The kernel does not forbid edits; it declines to promise anything about them.

4.5 Unfolding

Before time enters, the semantics of a design is *unfolding*: replace each reference to a realized declaration by its body, recursively, stopping at unresolved declarations (`Unfolds  $\Delta e e'$`). Let `DependsOn  $\Delta a b$` hold when the body of a refers to b .

Theorem 7 (Unfolding; `Unfolds.det`, `Unfolds.exists_of_acyclic`, `Unfolds.not_of_cyclic`, `Unfolds.refFree_of_fullyRealized`). Unfolding is deterministic; on the delay-free fragment it exists iff the reference graph is acyclic; a reference on a cycle through realized declarations has no unfolding at all; and a fully realized well-typed design unfolds to a reference-free program of the same type.

Acyclicity is witnessed by a rank that strictly decreases along edges, `Acyclic  $\Delta := \exists rank. \forall a b. \text{DependsOn } \Delta a b \rightarrow rank\ b < rank\ a$` , and excludes cycles (`Acyclic.not_cyclic`). The pure fragment has no fixpoints, so a cyclic definition denotes nothing; §6 shows which cycles become meaningful once delay exists, and that unfolding agrees with tick evaluation on the first-order fragment (Theorem 15).

5 Nominal concepts, representation and dimensions

Section 4 typed declarations without saying what a type means. This section adds the two things a product concept carries that a number does not — an identity that survives representation, and a physical dimension — and states the theorems that make the identity trustworthy.

5.1 Nominal identity and the grant

Suppose concepts were represented only by their representation types, so that `Tilt` and `MotorAngle` are both `q Angle`. Then the wire `motorTarget := tiltSensor` is well typed and the design is globally well formed, because nothing in the model records the distinction. Nominal types `sem s` over an internal identity record it: two distinct identities are

distinct types regardless of representation, so the invalid wire is rejected by T-App with no additional judgment. An explicit relationship `tiltToMotor : Tilt -> MotorAngle` is an ordinary declaration of arrow type — signature-first, possibly unresolved — and it appears in the term wherever a crossing occurs. The kernel has no cast, coercion or conversion.

Nominal identity alone leaves concept values opaque: under T-Ref and T-App only, a value of sem s can originate only in a declaration of semantic type (`no_semantic_value_without_declaration`). To let a formula realize a mapping, representation must be observable and constructible, and the obvious way to add it destroys what identity just bought. With global $\text{rep}_s : \text{sem } s \rightarrow R$ and $\text{mk}_s : R \rightarrow \text{sem } s$ available everywhere, $\lambda x. \text{mk}_{\text{Motor}}(\text{rep}_{\text{Tilt}} x)$ is a well-typed `Tilt -> MotorAngle` in the empty environment with no declaration and no mapping (`unrestricted_representation_binding_bypasses_semantic_identity`), and the crossing can hide inside a body whose signature mentions no motor (`hidden_crossing_inside_unrelated_body`). Observation alone is safe but cannot realize a mapping.

The grant separates the two. `rep` is typed everywhere (T-Rep); `mk s` is typed only where $G s$ (T-Mk); client code is typed under `Grant.none` and a body under `Grant.of` of its own signature (the definition of `Satisfies`). Let e . constructs s hold when `mk s` occurs in e .

Theorem 8 (Construction is granted; HasType.constructs_granted). If $\Theta; \Delta; G; \Gamma \vdash e : \tau$ and e . constructs s , then $G s$. Under `Grant.of` τ : a value of sem s is built only inside a realization whose signature announces sem s .

The hidden crossing above is rejected under the grant of an unrelated declaration and becomes legal, and visible, once `tiltToMotor` is declared (`hidden_crossing_rejected_under_grant`, `representation_binding_does_not_enable_hidden_semantic_mapping`).

Two constraints on representations in Θ . WF were not anticipated. Representations must be concept-free: if `Tilt` may be represented by `MotorAngle`, then `rep` itself is a hidden mapping under every policy including observation-only. And they must be data, a requirement that arrived from the reactive semantics: a concept value may be delayed, and a function-typed representation would carry a closure across ticks.

The grant is a known shape — the private constructor of an abstract type exported only to its defining module [4], or a capability attached to a definition site. What is specific is that the capability comes from the signature the designer already wrote, so no annotation is added, and one consequence follows: after all bodies are inlined into one program, that program is checked under the universal grant, because each construction was authorized at its own declaration. Semantic isolation is a property of the design graph and survives inlining as provenance (Theorem 14), not as a type property of the executable.

5.2 Erasure

Let $\rho : \text{SemanticId} \rightarrow \text{Ty}$ map each concept to a data type, agreeing with Θ on bound concepts. Erasure τ^ρ replaces sem s by ρs throughout a type; on terms, `rep e` and `mk s e` erase to e^ρ , and the type indices of operators are erased.

Theorem 9 (Erasure is sound; HasType.erase). If Θ . WF, ρ agrees with Θ , and $\Theta; \Delta; G; \Gamma \vdash e : \tau$, then $\Theta; \Delta^\rho; G'; \Gamma^\rho \vdash e^\rho : \tau^\rho$ for every grant G' .

Erasure is not injective — `Tilt` and `MotorAngle` erase to the same type (`erase_not_injective`) — and the untyped baseline is exactly what erasure leaves: the design the nominal calculus rejects is accepted after erasure (`baseline_is_erased_modelA`). Generated code is therefore ordinary code; the semantic layer has no runtime residue.

Three alternatives were formalized and refuted. A model in which the display name is the identity makes renaming destructive (`rename_under_name_identity_breaks_client`), whereas here a rename preserves identity (`semantic_rename_preserves_identity`). Treating a concept as an ordinary declaration admits category errors: the concept becomes usable as a value and can be realized by a number. Keeping identity out of the type as interface metadata checked by a direct-wire rule is evaded by η -expansion, since $(\lambda x. x)$ *tilt* has the same flow with no direct wire (`bweak_evaded_by_eta`); a compositional role judgment strong enough to close that gap has the rule shapes of typing over sem and duplicates it.

5.3 Dimensions

A physical quantity has type $q\ d$. There is no dimension-specific typing rule: $\text{add}_d : q\ d \rightarrow q\ d \rightarrow q\ d$, $\text{mul}_{d_1 d_2} : q\ d_1 \rightarrow q\ d_2 \rightarrow q\ (d_1 + d_2)$ and $\text{div}_{d_1 d_2}$ with $d_1 - d_2$ are registered operators, and an application is checked by T-App. $\text{length} + \text{time}$ is ill typed (`dimension_mismatch_rejected`); erasing every dimension to the zero vector is a sound translation that accepts it (`counterexampleB_baseline_accepts_length_plus_time`), so the untyped numeric baseline is the erasure of dimensional typing in the same sense that it is the erasure of nominal typing.

Dimension and identity are orthogonal. `Tilt` and `MotorAngle` both bound to $q\ \text{Angle}$ remain distinct types (`same_dimension_does_not_imply_same_semantic_identity`); a mapping realized by the dimensioned formula $\lambda x. \text{mk Brightness}(\text{rep } x \cdot \text{gain})$ with $\text{gain} : q\ (0 - \text{Angle})$ is typed, a dimension error inside it is caught by the same typing, and the formula cannot manufacture a `MotorAngle` despite the shared dimension (`explicit_semantic_mapping_uses_dimensioned_formula`). The association between a concept and its dimension lives in Θ , not in the identity and not in the type constructor.

Units are not in the calculus at all. A literal `90 deg` elaborates to $\text{lit}_{\text{Angle}}$ of a scaled magnitude; a coordinate $\text{inUnit}(q, u)$ is q divided by a scale constant and has dimension zero; with $\text{Unit}(x, u)$ is the converse; a conversion is their composition. Each is elaboration, none is a kernel construct, and a unit choice never reaches a type: `1 m` and `100 cm` are equal values of one type. The unit laws are proved above the kernel over an abstract scalar domain and instantiated exactly by a symbolic group in which π is a generator, so that a degree is exactly $\pi/180$ radian; we do not develop them here. Dimensional typing is thus Kennedy’s discipline [8], [9] without unit polymorphism in the kernel: dimension variables appear only in the surface’s definitional families (§8.4), where matching against closed dimensions instantiates them.

6 Reactive semantics in one domain

Every declaration denotes a stream over a tick domain. This section gives the single-domain semantics — one primitive, `delay` — and proves it deterministic and total on causal designs. Section 7 generalizes it to many domains and shows that this section is the diagonal of that one.

6.1 Values and evaluation

Values are booleans, naturals (which also carry every $q\ d$; the executable kernel’s magnitudes are naturals), tagged concept values `sem s v`, `none`, `some v`, lists, pairs, closures `clo ρ e` over a value environment, and partially applied operators `prim p v̄`. An operator is computed when saturated: `applyPrim p v̄` is `compute p v̄` if $|v̄|$ equals p ’s arity and `prim p v̄` otherwise.

The judgment $\rho \vdash_t e \Downarrow v$ — the value of e at tick t under local environment ρ , with the design Δ and the input I ambient — is defined in Figure 4 (Ev).

$$\begin{array}{c}
\frac{\rho(x) = v}{\rho \vdash_t x \Downarrow v} \quad \frac{}{\rho \vdash_t \lambda x : \tau. e \Downarrow \text{clo } \rho e} \quad \frac{}{\rho \vdash_t p \Downarrow \text{applyPrim } p []} \\
\frac{\rho \vdash_t f \Downarrow \text{clo } \rho' b \quad \rho \vdash_t a \Downarrow w \quad w :: \rho' \vdash_t b \Downarrow v}{\rho \vdash_t f a \Downarrow v} \quad \frac{\rho \vdash_t f \Downarrow \text{prim } p \vec{u} \quad \rho \vdash_t a \Downarrow w}{\rho \vdash_t f a \Downarrow \text{applyPrim } p (\vec{u} ++ [w])} \\
\frac{\Delta^{\text{real}}(\delta) = \text{some } b \quad [] \vdash_t b \Downarrow v}{\rho \vdash_t \text{declRef } \delta \Downarrow v} \text{ (E-Real)} \quad \frac{\Delta^{\text{real}}(\delta) = \text{none}}{\rho \vdash_t \text{declRef } \delta \Downarrow I \delta t} \text{ (E-Input)} \\
\frac{\rho \vdash_t e \Downarrow \text{sem } s w}{\rho \vdash_t \text{rep } e \Downarrow w} \quad \frac{\rho \vdash_t e \Downarrow w}{\rho \vdash_t \text{mk } s e \Downarrow \text{sem } s w} \\
\frac{\rho \vdash_0 i \Downarrow v}{\rho \vdash_0 \text{delay } i e \Downarrow v} \text{ (E-Delay0)} \quad \frac{\rho \vdash_t e \Downarrow v}{\rho \vdash_{t+1} \text{delay } i e \Downarrow v} \text{ (E-DelayS)} \\
\frac{\rho \vdash_t f \Downarrow v_f \quad \rho \vdash_t z \Downarrow v_z \quad \rho \vdash_t l \Downarrow \text{list } []}{\rho \vdash_t \text{fold } f z l \Downarrow v_z} \text{ (E-FoldNil)}
\end{array}$$

$$\frac{\begin{array}{c} \rho \vdash_t f \Downarrow v_f \quad \rho \vdash_t z \Downarrow v_z \quad \rho \vdash_t l \Downarrow \text{list}(x :: xs) \\ [\text{list } xs, v_z, v_f] \vdash_t \text{fold } \#2 \#1 \#0 \Downarrow r \quad [r, x, v_f] \vdash_t \#2 \#1 \#0 \Downarrow v \end{array}}{\rho \vdash_t \text{fold } f z l \Downarrow v} \text{ (E-FoldCons)}$$

Figure 4. Single-domain evaluation (Ev), with Δ and I ambient. In one domain sync c evaluates exactly as delay (rules *syncZero*, *syncSucc*), which §7 justifies. Literals evaluate to themselves. $\#i$ is de Bruijn index i .

Three points of Figure 4 deserve comment. An unresolved declaration is an *input*: E-Input reads $I \delta t$. A realized declaration is evaluated from its body at the current tick in the *empty* environment (E-Real): a reference’s value never depends on the local environment of the reader, which is what makes a declaration a stream rather than a function of its call site (Ev.declRef_env_irrelevant). And delay shifts the tick: read at $t + 1$, it evaluates its operand at t ; at 0 it evaluates the initial value.

The recursor’s rule unrolls syntactically. Rather than a recursive definition of a fold on values, the rule evaluates the syntactic term `fold #2 #1 #0` in an environment holding the tail, the seed and the function, and then the term `#2 #1 #0` in an environment holding the result, the head and the function. This keeps Ev an ordinary inductive relation with no mutual recursion, so every proof by induction on Ev that predated the recursor extends by one case, and totality is a separate lemma by induction on the list (§8.1).

Theorem 10 (Determinism; Ev.det). If $\rho \vdash_t e \Downarrow v_1$ and $\rho \vdash_t e \Downarrow v_2$ then $v_1 = v_2$.

Evaluation is a partial function with no hidden evaluation order — there are no effects to order — and this holds unconditionally.

An executable interpreter `evalF` with a fuel parameter is proved sound for the relation (`evalF_sound`, `Ev.of_evalF`). Every trace in the development and in this paper was computed by it inside the proof checker.

6.2 Cycles and causality

Let e .instRefs be the declarations e refers to *instantaneously*: those not under the delayed operand of a delay or sync (the initial value is read at tick 0 and counts as instantaneous). `InstDependsOn  $\Delta$  a b` holds when $b \in \text{instRefs}$ of a ’s body.

Definition (Causal). `Causal  $\Delta$` := $\exists \text{rank } R. (\forall \delta. \text{rank } \delta < R) \wedge \forall a b. \text{InstDependsOn } \Delta a b \rightarrow \text{rank } b < \text{rank } a$.

On the delay-free fragment `InstDependsOn` is `DependsOn`, so causality is exactly bounded acyclicity (`Causal iff_acyclic_of_delayFree`): the earlier condition is the timeless special case rather than a replaced requirement. A structural cycle every path of which passes through a delayed operand — $A := \text{delay } \emptyset B$; $B := A$, or a self-delayed accumulator — is causal. A cycle that is partly delayed is not.

Theorem 11 (Strict cycles have no value; Ev.not_of_strictCyclic). If a lies on a cycle of references passing through neither a delayed operand nor a lambda, then for every tick and environment there is no v with $\rho \vdash_t \text{declRef } a \Downarrow v$.

Not “some default”, not “one of several”: no derivation exists. A gap should be recorded. A cycle guarded by a lambda, $A := \lambda x. A x$, is rejected by Causal yet `declRef A` does evaluate — to a closure; only applying it diverges. Causal is conservative for lambda-guarded cycles and Theorem 11 covers strict cycles only.

6.3 The logical relation and totality

Totality is proved by a logical relation indexed by the tick. The relation is stated generically in an *application relation* $A : \text{Value} \rightarrow \text{Value} \rightarrow \text{Value} \rightarrow \text{Prop}$ — how a function value applied to an argument yields a result — so that the single-domain and multi-domain semantics share one relation. At a fixed tick, A is `Apply  $\Delta I t$` : v_f applied to w yields v when v_f is a closure whose body evaluates to v at t under w , or a partial operator whose saturation is v .

$$\begin{aligned}
\mathcal{R}_\Theta^A[\text{bool}] v &\Leftrightarrow \exists b. v = \text{bool } b \\
\mathcal{R}_\Theta^A[\text{nat}] v &= \mathcal{R}_\Theta^A[\text{q } d] v \Leftrightarrow \exists n. v = \text{nat } n \\
\mathcal{R}_\Theta^A[\text{opt } \tau] v &\Leftrightarrow v = \text{none} \vee \exists w. v = \text{some } w \wedge \mathcal{R}_\Theta^A[\tau] w \\
\mathcal{R}_\Theta^A[\text{list } \tau] v &\Leftrightarrow \exists \vec{w}. v = \text{list } \vec{w} \wedge \forall w \in \vec{w}. \mathcal{R}_\Theta^A[\tau] w \\
\mathcal{R}_\Theta^A[\tau \times \sigma] v &\Leftrightarrow \exists x y. v = \text{pair } x y \wedge \mathcal{R}_\Theta^A[\tau] x \wedge \mathcal{R}_\Theta^A[\sigma] y \\
\mathcal{R}_\Theta^A[\tau \rightarrow \sigma] v &\Leftrightarrow \forall w. \mathcal{R}_\Theta^A[\tau] w \rightarrow \exists v'. A v w v' \wedge \mathcal{R}_\Theta^A[\sigma] v' \\
\mathcal{R}_\Theta^A[\text{sem } s] v &\Leftrightarrow \exists w. v = \text{sem } s w \wedge \forall R. \Theta s = \text{some } R \rightarrow \mathcal{R}^A[R] w
\end{aligned}$$

The concept clause says that a concept value is a tagged representation value. Its inner use of the relation at the representation R is the concept-free relation RedSF ; because Θ . WF makes R concept-free, the definition is well founded on the type without appeal to Θ (Red_semFree). At data types the relation is independent of A (Red_data), which is what allows a delayed value to be transported between ticks. Registered operators are related at their types for any A that saturates them (Red_prim).

Well-typed inputs are inputs related to the type view: $\Delta^{\text{ty}}(\delta) = \text{some } \tau \wedge \Delta^{\text{real}}(\delta) = \text{none} \Rightarrow \mathcal{R}_\Theta^{\text{Apply } \Delta I t}[\tau] (I \delta t)$ for every t .

Theorem 12 (Fundamental theorem; fundamental). Let Θ . WF, let rank, R witness Causal Δ , let $\text{GlobalWF } ev \Theta \Delta$ and let I be well typed. Then for every tick t , bound r , grant G , and $\Theta; \Delta; G; \Gamma \vdash e : \tau$, and every ρ related to Γ such that every instantaneous reference of e has rank below r , there is v with $\rho \vdash_t e \Downarrow v$ and $\mathcal{R}_\Theta^{\text{Apply } \Delta I t}[\tau] v$.

Proof sketch. Lexicographic induction on $(t, r, \text{derivation})$. A delayed operand at tick $t + 1$ is evaluated at tick t under *any* rank (the first component decreases); an instantaneous reference to a realized declaration δ is evaluated at the same tick under the smaller bound $\text{rank } \delta$ (the second decreases), and its body is well typed under the grant of its own signature by GlobalWF ; every other case is the induction on the derivation. The fold case uses fold_total (§8.1). ◻

Theorem 13 (Totality; reactive_total, Ev.red). In a causal, globally well formed design with well-typed inputs, every declared identity has a value at every tick, and that value — unique by Theorem 10 — is related to its expected type.

The relation is a step-indexed logical relation in the sense of Appel and McAllester [6] and Ahmed [7], with the tick as the index and the rank as a second, inner index; what differs is that the index counts *time* rather than *steps*, so that the induction on it is the induction that makes a delayed self-reference well defined.

6.4 Two restrictions forced by totality

T-Delay and T-Sync restrict their type to data and their context to empty. Neither restriction was a design decision; each is what the induction of Theorem 12 needs. A delayed closure would be a value at tick t related by $\mathcal{R}^{\text{Apply } \Delta I t}$ that must be transported to tick $t + 1$, and the arrow clause is tick-indexed and cannot be transported; Red_data is exactly the statement that data clauses can. A delay under a binder would evaluate its operand at the previous tick in an environment created at the current one. The restrictions have two corollaries stated as theorems: nothing of function type can be delayed or transported ($\text{arrow_not_delayable}$, by inversion), and memory and transport are typed only in the empty context ($\text{delay_not_under_binder}$, $\text{sync_not_under_binder}$). Temporal state therefore belongs to declarations, and a reusable stateful component is instantiated into fresh declarations (§10) rather than abstracted over.

Initialization is semantic, not validation. Every delay carries an explicit initial value. Two toy relations without one show why: the first tick is either undefined ($\text{first_tick_undefined_without_init}$) or nondeterministic ($\text{first_tick_nondeterministic_without_init}$).

6.5 Provenance through time

State carries semantic tags; it never creates them. Let v . Taints s hold when the tag s occurs anywhere inside v — including inside closures’ environments and bodies.

Theorem 14 (Tag provenance; `Ev.tag_provenance`, `temporal_state_preserves_semantic_identity`). If no realization in Δ constructs s , no input value is tainted by s , e does not construct s and ρ is clean, then every value $\rho \vdash_t e \Downarrow v$ is clean. In particular a delayed value carries exactly the tag of the value delayed.

Combined with Theorem 8 this is the runtime half of semantic isolation: a concept appears in a value only if some signature announces it or some input carries it, at every tick. The typing rule `delay` : $\tau \rightarrow \tau \rightarrow \tau$ at data τ gives the static half — a delayed tilt is a tilt, and a backward difference over a time step has dimension `Length` — Time with no derivative primitive.

6.6 Wiring designs and unfolding

The first-order fragment of interest to a compiler consists of *wiring* terms — references, literals, operators, applications, `rep`, `mk`, `delay` and `sync`, with no lambda — and designs all of whose bodies are wiring terms. On this fragment closures never arise (`Ev.noClo`) and evaluation is independent of the local environment (`Ev.env_irrelevant`).

Theorem 15 (Unfolding preserves stepping; `unfolds_preserves_eval`). On a wiring design with closure-free inputs, if `Unfolds` $\Delta e e'$ then $\rho \vdash_t e \Downarrow v$ iff $\rho \vdash_t e' \Downarrow v$.

This licenses inlining: the value of the unfolded program at a tick is the value of the referencing program. A more general statement is available for *pure* terms — closed terms with no reference, state or transport — whose value is the same in every design, at every tick, under every input (`Ev.pure`); §8.4 uses it for the definitional library.

6.7 Derived temporal operators

Every temporal operator a surface language offers reduces to `delay` and registered operators. Table 2 lists the elaborations; each is a declaration referring to itself — a self-delayed cycle, the class that structural acyclicity forbade and causality licenses — and each was typed, checked causal, and run on a concrete trace. There is no independent kernel definition of `count` for the reduction to be proved equal to; the claim is that the elaboration has the intended trace.

surface	declaration body
<code>previous x</code>	<code>delay init x</code>
<code>previous x</code> without an initial value	<code>delay none (some x)</code> ; the absence is pushed to consumers
<code>hold init e</code>	<code>getD e (delay init self)</code>
<code>count e</code>	<code>ite (isSome e) (1 + delay 0 self) (delay 0 self)</code>
<code>since e</code>	<code>ite (isSome e) 0 (1 + delay 0 self)</code>
<code>once e</code>	<code>delay false self ∨ isSome e</code>
<code>every n</code>	a modulo- n counter over <code>delay</code>
<code>rise b</code>	$b \wedge \neg \text{delay false } b$, as an optional Boolean

Table 2. Derived temporal operators (*Experiments/ReactiveAlternatives.lean*).

There is no signal type in Ty: under this semantics a signal type would be inhabited by exactly the terms of the underlying type and would reject nothing. There is no event type: within one domain an input delivers at most one value per tick by construction, so an occurrence is a stream of optional type, and the streams of type `opt` τ are exactly the streams of multiplicity at most one. What separates an occurrence from an optional value can only be seen when a source ticks faster than its observer, which is a cross-domain question (§7.5). State has no identity of its own: a cell is a delay in a declaration body, consumers refer to the declaration, and there is consequently no notion of two writers to one cell.

7 Clock domains

“Contact and orientation move with the interaction; temperature moves with the environment.” A designer can say this before any rate is known, and it is a statement about which quantities update together, not about how often. The calculus records it as a nominal **clock domain** and treats rate as data outside the kernel.

7.1 Schedules and the domain judgment

The time model is one global base tick and a schedule $S : \text{ClockId} \rightarrow \mathbb{N} \rightarrow \text{Bool}$ saying at which global ticks each domain activates. A period n induces the schedule $t \bmod n = 0$ (`Sched.periodic`); the schedule lives outside the design. Domain-local time is not a separate counter but the sequence of a domain’s activations. The last activation of c strictly before t is

$$\text{prevAct } S \ c \ 0 = \text{none}, \quad \text{prevAct } S \ c \ (t + 1) = \text{if } S \ c \ t \text{ then some } t \text{ else prevAct } S \ c \ t,$$

with $\text{prevAct } S \ c \ t = \text{some } t' \Rightarrow t' < t \wedge S \ c \ t'$.

Each declaration is assigned a domain by the clock environment K , or none if it is a pure mapping usable anywhere. The clock is interface data in every sense that matters — clients’ validity depends on it, it is frozen under refinement, and changing it is an edit (Table 1) — and it is stored as a projection beside the interface, as a concept’s representation is stored in Θ rather than in the type.

The **domain judgment** $\text{Clocked } K \ c \ e$, for $c : \text{Option ClockId}$, says that e may be evaluated in domain c :

$$\begin{aligned} \text{Clocked } K \ c \ (\text{declRef } \delta) &\Leftrightarrow K \ \delta = \text{none} \vee K \ \delta = c \\ \text{Clocked } K \ (\text{some } c) \ (\text{delay } i \ e) &\Leftrightarrow \text{Clocked } K \ (\text{some } c) \ i \wedge \text{Clocked } K \ (\text{some } c) \ e \\ \text{Clocked } K \ (\text{some } c) \ (\text{sync } c' \ i \ e) &\Leftrightarrow \text{Clocked } K \ (\text{some } c) \ i \wedge \text{Clocked } K \ (\text{some } c') \ e \\ \text{Clocked } K \ \text{none} \ (\text{delay } i \ e) &\Leftrightarrow \text{False} \qquad \text{Clocked } K \ \text{none} \ (\text{sync } c' \ i \ e) \Leftrightarrow \text{False} \end{aligned}$$

and homomorphically elsewhere. A reference stays in its domain or is agnostic; a delay needs a domain; `sync` c' switches the domain of its operand. A design is well clocked when every body is clocked in its own declaration’s domain. Typing is unchanged and blind to domains: the direct wire between two domains at the same value type is well typed and rejected only by `Clocked`. Placing the domain in the type instead was tried and set aside: every pure mapping would then need clock polymorphism (`clocked_type_forces_polymorphism`), and nothing the type rejects is missed by the judgment.

7.2 Multi-domain evaluation

The judgment $\rho \vdash_t^c e \Downarrow v$ — in domain c at global tick t , with S, Δ, I ambient — is `Ev` with the two temporal rules replaced by four (`MEV`):

$$\begin{array}{c} \frac{\text{prevAct } S \ c \ t = \text{none} \quad \rho \vdash_t^c i \Downarrow v}{\rho \vdash_t^c \text{delay } i \ e \Downarrow v} \qquad \frac{\text{prevAct } S \ c \ t = \text{some } t' \quad \rho \vdash_{t'}^c e \Downarrow v}{\rho \vdash_t^c \text{delay } i \ e \Downarrow v} \\[10pt] \frac{\text{prevAct } S \ c' \ t = \text{none} \quad \rho \vdash_t^c i \Downarrow v}{\rho \vdash_t^c \text{sync } c' \ i \ e \Downarrow v} \qquad \frac{\text{prevAct } S \ c' \ t = \text{some } t' \quad \rho \vdash_{t'}^{c'} e \Downarrow v}{\rho \vdash_t^c \text{sync } c' \ i \ e \Downarrow v} \end{array}$$

delay reads the previous activation of the current domain; `sync` c' reads the previous activation of c' and evaluates its operand *there*, in c' . All other rules carry c unchanged.

Theorem 16 (Memory is transport at the own domain; `delay_is_sync_own`, `clocked_delay_iff_sync_own`). $\rho \vdash_t^c \text{delay } i \ e \Downarrow v$ iff $\rho \vdash_t^c \text{sync } c \ i \ e \Downarrow v$, and $\text{delay } i \ e$ is clocked in c iff $\text{sync } c \ i \ e$ is.

The kernel therefore has one temporal primitive — read a domain at its previous activation — and delay is notation for its diagonal. A delay in a slow domain reads three global ticks back where a delay in a fast one reads one, with the same syntax.

Theorem 17 (Single-domain embedding; `single_domain_embedding`). Under the always-active schedule, $\rho \vdash_t^c e \Downarrow v$ iff $\rho \vdash_t e \Downarrow v$, for every c .

The results of §6 are thus the one-domain special case of this section rather than a replaced machine.

Theorem 18 (Determinism; MEv.det). Multi-domain evaluation is a partial function, for every schedule.

Theorem 19 (Totality in every domain; mfundamental, multi_domain_total). In a causal, globally well formed design with inputs well typed in every domain, every declared identity has a value in every domain at every tick, related to its expected type.

The proof reuses the logical relation of §6.3 with the application relation $\text{MApply } S \Delta I c t$, and the same lexicographic induction: a transport at t evaluates its operand at $t' < t$ under any rank. Causality is the *same* Causal Δ : a transport’s operand is never instantaneous, so no cross-domain cycle can be. An interpreter mevalF is proved sound (mevalF_sound). Tag provenance holds across domains ($\text{MEv.tag_provenance}$): transport changes timing, not identity, and a crossing from Tilt@fast to Tilt@slow authorizes neither $\text{Tilt} \rightarrow \text{MotorAngle}$ nor $q \text{ Length} \rightarrow q \text{ Time}$, by the typing rule.

7.3 Strictly before, and what the alternative exposes

A transport sees only source activations strictly before the destination tick. That is a choice with an observable alternative, and the alternative was built.

Theorem 20 (Same-tick visibility exposes the scheduler; scheduling_order_observable). Let $\text{MEv}_{\leq}$ be the semantics in which a transport may also see a simultaneously active source, resolved by a priority between domains. There is a two-domain design, a schedule and an input such that two priorities give two different values to the same declaration at the same tick.

At tick 1 both domains are active for the first time; with the source first the transport delivers the source’s current value, with the destination first it delivers the initial value. The strictly-before rule has no such parameter, and Theorem 18 has no order between simultaneously active domains in its statement. Every crossing costs one destination-visible step; “synchronous sub-domains evaluated in one instant” are, in this model, the same domain.

Rate and identity are distinct. A clone of a domain with the identical schedule is a different domain, and a direct wire between them is rejected ($\text{equal_rate_not_same_domain}$); a domain at the same rate but shifted in phase reads different values through a transport. Rate changes are validation-only: they change the induced schedule and the observed values, but no client’s well-formedness. This is where the calculus departs from synchronous languages that recover clocks by inference [10], [11]: the domain is authored, because the information needed to infer it — the realization binding — arrives at the point where a designer is least able to make the decision.

7.4 Occurrences across domains: the window buffer

Within one domain an occurrence is a stream of optional type. Across domains this fails: sync is a zero-order hold, so a slow consumer of a fast event source sees the last value only. Two fast events at ticks 1 and 2 and one event at tick 2 are indistinguishable at the slow activation at tick 3, and a single event at tick 1 followed by a quiet fast tick is dropped outright ($\text{opt_loses_multiplicity_under_sync}$). The counterexample is against sync as an *event transport*, not against optional types; it says that multiplicity and order are observable across domains and that keeping them requires buffering.

What the destination should see is the source’s activations since the destination’s own previous activation — the *window*, $\text{windowTicks } S \text{ src dst } t$, the source ticks in $[\text{prevAct } S \text{ dst } t, t)$. The window equals the source’s accumulated log read at the current tick minus its length at the previous destination activation ($\text{buffer_from_log_and_cursor}$): two single-instant reads, a sync of a source-side accumulator and a delay of a cursor. With list data this is five ordinary declarations:

```
log    @src : cons src (delay nil log)      -- source-side accumulator
logD   @dst : sync src nil log              -- the log, transported
seen   @dst : length logD                  -- log length now
cursor @dst : delay 0 seen                  -- log length at the previous activation
window @dst : reverse (take (seen - cursor) logD) -- the new entries, oldest first
```

Theorem 21 (The window is computed; buffer_window_correspondence). For every schedule, input, destination domain and tick, if the five declarations are realized as above and src is an input, then $[] \vdash_t^{dst} \text{window} \Downarrow \text{list}(\text{map}(I\ src)(\text{windowTicks } S\ src\ dst\ t))$.

The elaboration is well typed and well clocked (`buffer_elaboration_well_typed`, `buffer_elaboration_well_clocked`); the list is injective on windows, ordered by tick, and multiplicity-preserving for every predicate (`buffer_lossless`, `window_to_list_preserves_order`, `window_to_list_preserves_multiplicity`). The general negative result is that any summary depending only on the newest k entries, for any fixed k , identifies a k -entry window with a $(k + 1)$ -entry window (`bounded_summary_not_lossless`); a lossless summary is injective and therefore unbounded (`lossless_iff_injective`). Capacity is thus a deployment obligation on the schedule — for periodic schedules one destination period of source activations suffices (`periodic_capacity_sufficient`) — and the only overflow policy that preserves the semantics is to reject the deployment; dropping is a semantic change (`bounded_buffer_agrees`, `negE`).

The point for the calculus is what was *not* added: no event type, no buffer primitive, no scheduler order, no same-tick visibility, no implicit overflow rule. An event stream is a data-typed declaration in a domain; an occurrence is its value at an activation; a lossless view of it across domains is the five declarations; latest, count, coalesce are ordinary computations over `window`.

8 Data, the recursor and polymorphism

The kernel of §3–7 computes with booleans, counts, quantities, concepts and optional values and abstracts with lambdas over them. A product needs more: a mode tested against a finite set of modes, every reading below a threshold, a pair of readings, a calibration mapped over a collection. This section records the smallest typed data basis that supports them, what could not be derived and why, and how polymorphism is provided without a type variable in the kernel.

8.1 The list recursor

`fold f z l` is a *term former*, not a registered operator. The kernel has no recursion, deliberately; a total language needs an eliminator for its inductive data, and `fold` is the one construct that applies a function value in the course of evaluation. Registered operators never apply closures. The alternative of one primitive per collection operation was rejected because a primitive cannot apply a closure and each would need its own evaluation rule; the alternative of bounded unrolling was rejected because lists — the cross-domain window — are unbounded.

Theorem 22 (The recursor is total; fold_total, mfold_total). If v_f is related at $\tau \rightarrow \sigma \rightarrow \sigma$, v_z at σ and every element of $\vec{w}$ at τ , then $[\text{list } \vec{w}, v_z, v_f] \vdash_t \text{fold } \#2\ \#1\ \#0 \Downarrow r$ for some r related at σ .

The proof is an induction on the list, separate from the fundamental theorem, which invokes it in its fold case. Every collection operation — `map`, `filter`, `any`, `all`, `contains`, `append`, `sum`, `zip`, and through `toList` the option eliminators — is a definition over `fold`, and each is proved to compute the mathematical function it names through one general lemma: the recursor computes `List.foldr g` whenever the step closure implements g on the reachable accumulators (`fold_spec`; then `any_spec`, `all_spec`, `map_spec`, `filter_spec`, `min_spec`, `clamp_spec`, ...). Finite quantification is a fold — $\forall x \in xs. Px$ iff `all xs P` evaluates to true (`forall_in_list`, `exists_in_list`) — and a finite-set literal means membership with duplicates irrelevant (`oneOf_mem`, `oneOf_dup_irrelevant`), so there is no `Set` type and no uniqueness convention.

8.2 Products, and why not Church pairs

$\tau \times \sigma$ with `pair`, `fst`, `snd` entered the kernel after the Church encoding was tried and refuted twice. A Church pair is an arrow, and arrows are not data: nothing of function type can be delayed or transported (`arrow_not_delayable`), so paired *state* — a delayed reading with its timestamp — needs a data product. And a Church pair used as a first-class value needs rank-2 types: in a toy System F with a rank measure, the type of `fst` on Church pairs has rank 2 (`church_fst_rank`), and in the prenex fragment a pair instantiated at one result type serves only one projection (`church_pair_prenex_one_projection`). Products are value composition only; they are never a component interface or an output bundle (§10 shows what a tuple-returning declaration does to the dependency graph).

8.3 Equality with its evidence in the syntax

eq_τ^h is structural equality at every data type — booleans, numbers, none/some, pairs and lists componentwise, concept values by tag and representation — with the proof $h : \tau$. Data carried *in the syntax*. This is the kernel’s only capability evidence: an equality on a function type is unwritable rather than ill typed, which keeps T-Prim unconditional. On first-order values structural equality is equality (value.beq_iff , by a mutual induction over the nested value type).

Order is deliberately not generalized. A first formulation gave $<$ a structural meaning at every data type — booleans, options, pairs and lists lexicographically — and it was formally consistent. An audit rejected it on the grounds that no such order has a design meaning: $\text{mode1} < \text{mode2}$ would order modes by a constructor tag, $\text{None} < \text{Some } x$ is an artifact. The structural order was deleted and lt_d restored to quantities only. So $\text{Data} \Rightarrow \text{Eq}$ holds (Cap.eq_iff_data) but $\text{Eq} \not\Rightarrow \text{Ord}$; order on a *concept* is a surface capability — a concept the designer declared ordered and represented by a quantity compares as lt_d on rep, a term the kernel already admits ($\text{lt_only_on_quantities}$, lt_rejected , min_mode_rejected). Enumerations follow the same rule: equality is natural, declaration order is never silently behavioral order.

8.4 Rank-1 polymorphism by families

Five models of polymorphism were compared: a monomorphic kernel; per-type duplication; rank-1 parametric polymorphism; System F; higher rank. The one adopted is rank-1 *as definitional families*: every library entry is a function $\text{Ty} \rightarrow \text{Expr}$ (or $\text{Dim} \rightarrow \text{Expr}$) in the metalanguage, and a scheme is a pattern over type and dimension variables with capability constraints. The kernel sees only the instances ($\text{instances_are_monomorphic}$: three uses of min are three kernel terms), and T-Prim, T-App and Theorem 1 are unchanged.

Why this needs no kernel support: a use site always has *closed* argument types. Every declaration’s expected type is frozen and closed, and inference is bottom-up, so finding the instance of a scheme is one-way *matching* of the scheme’s pattern against closed types — decidable, returning the unique substitution on the pattern’s variables (matchTy_sound , matchTy_complete). There is no unification of two open types, no let-generalization inside expressions [12], and no principal-type search; those problems arise when a definition’s type is inferred from its body, and every definition here carries its signature. The situation is that of local type inference [13] with no bidirectionality needed. Capability constraints are checked after matching ($\text{Scheme.instantiate_sound}$), and the two failure points have designer-level explanations: *no instance* and *capability failed*. System F was rejected by measuring what it would add — the prenex fragment *is* instantiation of families — and every candidate higher-rank use has a rank-1 replacement (applyBoth_rank , $\text{applyBoth_replacement}$). Dimension polymorphism ($\text{sum} : \text{list } (q \ d) \rightarrow q \ d$) uses the same mechanism with dimension pattern variables; no kind system, because the dimension algebra already lives in the operator table.

Nominality survives all of it. *Any* family typed at $\alpha \rightarrow \alpha \rightarrow \alpha$, instantiated at concept s , rejects an argument of concept $s' \neq s$, the representations never consulted ($\text{generic_preserves_identity}$); the same for $q \ d$ versus $q \ d'$ ($\text{generic_preserves_dimension}$). This is Reynolds’s abstraction [14] and Wadler’s free theorems [15] at the level of syntax: a family cannot inspect what it is instantiated at, because it is instantiated by substitution into a closed term.

8.5 The library as combinators, and inlining

Every library entry is a **combinator**: variables, literals, lambdas, applications, registered operators, the recursor and rep — no reference, no state, no transport, no mk. For combinators four facts are proved once and combine into an inlining statement (lib_expansion): typing is independent of the design and the grant and reads Θ only through write-once bindings ($\text{HasType.comb_irrelevant}$); the value is the same in every design at every tick under every input ($\text{lib_eval_context_free}$, from Ev.pure); the term is clocked in every domain (lib_clocked); nothing is constructed (Comb.noConstruct). This is what lets an implementation inline an equation at each use without creating a declaration — a library entry as a declaration would be monomorphic and would enter the dependency graph.

The expressiveness ceiling, stated once: total first-order-data computation over booleans, quantities, concepts, options, lists and pairs, with higher-order functions and one list recursor; generic definitions instantiated at closed types; no general recursion, no type abstraction in terms, no sums (an enumeration with a payload is encoded as

a tag paired with an optional payload, and a kernel sum would cost one more eliminator term former exactly like fold), no unbounded quantification. This is a design conclusion backed by executed cases and the proved library; it is not a minimality theorem.

9 Physical outputs

A declaration computes a value; it does not move hardware. Physical effect happens only through an explicit **drive edge** from a declaration to a nominally identified **output** — a logical actuator channel, a resource in a different sort from both concepts and declarations: “the desired steering angle” is a value, “the steering motor” is a resource.

- $\Omega : \text{OutputId} \rightarrow \text{Option } \langle \text{accepts}, \text{clock} \rangle$ — each output’s accepted type and domain;
- $\beta : \text{DeclId} \rightarrow \text{Option OutputId}$ — the drive edges, a write-once per-declaration projection of the same shape as K ;
- $\text{DriveWF } \Omega \ K \ \Delta \ \beta := \forall \delta \ o. \ \beta \ \delta = \text{some } o \rightarrow \exists \text{spec}. \ \Omega \ o = \text{some spec} \wedge \Delta^{\text{ty}}(\delta) = \text{some spec.accepts} \wedge K \ \delta = \text{some spec.clock};$
- $\text{SingleDriver } \beta := \forall \delta_1 \ \delta_2 \ o. \ \beta \ \delta_1 = \text{some } o \rightarrow \beta \ \delta_2 = \text{some } o \rightarrow \delta_1 = \delta_2;$
- $\text{CompleteOutputs } \beta \ \text{req}$ — every required output is driven.

Nothing was added to types, typing, the domain judgment, evaluation or the grant. The edge neither coerces nor converts nor synchronizes: the driver’s type *equals* the accepted type and its domain *is* the output’s. A declaration typed `Tilt` cannot drive a `MotorAngle` output; an output that accepts a representation type needs an explicit rep-typed declaration in front of it; a slow driver reading a fast value must sync it upstream. A driver of a concept-accepting output is necessarily a value, not a function (`driver_is_unit_domain`).

Definition. $\text{PhysicalOutput } S \ \Delta \ I \ \Omega \ \beta \ o \ t \ v := \exists \delta \ \text{spec}. \ \beta \ \delta = \text{some } o \wedge \Omega \ o = \text{some spec} \wedge [] \vdash_t^{\text{spec.clock}} \text{declRef } \delta \Downarrow v.$

Theorem 23 (One driver, one output; single_driver_output_deterministic, multiple_direct_drivers_rejected). Under `SingleDriver` β , `PhysicalOutput` is a partial function of o and t . Two declarations driving one output — each well typed, well clocked, causal and individually well formed — violate `SingleDriver` and nothing else, and there is a tick at which the output receives two values.

The principle is *many contributors, one explicit final driver*. Contributors are dependencies: `base + corr -> final -> motor` passes every check; priority is a conditional in the single driver; blend, maximum and clamp are ordinary declarations of the target type. Why arbitration must be explicit is shown rather than argued: first-wins, last-wins and maximum over the same value graph give three different physical outputs (`hidden_arbitration_observable`). Binding an unbound declaration to an undriven output is a refinement and preserves `SingleDriver` (`first_output_binding_is_monotone`); binding to a driven output is invalid; retargeting, renaming or detaching an edge invalidates an unchanged design.

Two alternatives were formalized in toy form. Direct effect rows — the set of outputs a declaration drives — are exactly the drive edges, and single-driver is exactly their pairwise disjointness; propagated rows flag a valid design in which a display reads the driver; action values move the conflict into the collector that consumes them, which must then be a policy, which is the single driver by another name. None of this bears on richer effect systems [16]; it says these formulations add no rejection the single-driver rule lacks.

Finally, the dual form. Once zero-input relationships have the canonical interface type $() \rightarrow A$ (normalized above the kernel to A , with the unit eliminated before any term is typed — the kernel has no unit type, and `delay` inside a zero-input declaration is why: a unit binder would forbid memory there, `delay_not_under_binder`), the form $A \rightarrow ()$ suggests itself as a consumer. It cannot name one: in a pure total language every function into the one-point type is the same function (`unit_codomain_collapse`, by function extensionality), so two “consumers” are indistinguishable (`consumers_indistinguishable`), and the evaluation relation has no effect component (`eval_independent_of_drives`). Naming a receiver needs an output semantics, and the calculus already has exactly one.

10 Composition by renaming

A second lamp should reuse the first’s behavior without copying it. This needs a component with a promised interface, instantiated with fresh identity and bound to other behaviors. The section’s result is that all of it is derivable from what §4 provides — realization plus renaming — and that the flattened system is checked by the unchanged judgments.

10.1 Equivariance

A renaming r bundles four maps — on declaration, semantic, clock and output identities. Renaming acts on types (through `sem`), on terms, on interfaces, on declarations and pointwise on environments; Δ . `RenamedBy` r Δ' says Δ' stores the renamed declaration of Δ at the renamed identity.

Theorem 24 (Equivariance; `HasType.rename`, `Satisfies.rename`, `Clocked.rename`). If $\Theta; \Delta; G; \Gamma \vdash e : \tau$ and Θ', Δ', G' are the images of Θ, Δ, G under r (agreement on the image, with no injectivity required), then $\Theta'; \Delta'; G'; \Gamma \vdash e^r : \tau^r$; likewise for satisfaction, and for the domain judgment under a clock environment that agrees on the declared identities.

Evidence must be equivariant as well (`Evidence.Equivariant`), an abstract condition beside monotonicity. Nothing else is new in the composition theory; the rest is definitions over Theorem 24 and §4.

10.2 Components, instances and flattening

A **port** is a template declaration by local identity with the public part of its interface and its parameter clock. A **behavior interface** has required ports (unresolved declarations a composer binds), provided ports, elaboration-time parameters (unresolved data-typed declarations bound to closed constants at instantiation) and clock parameters. A **component** is an interface, a template design over local identities below a width W , and a partition of its concepts and outputs into private (freshened per instance) and shared. Realizes ev C is a predicate over the existing judgments: the template is a well-formed design (`Design.WF`: `GlobalWF`, Θ . `WF`, well clocked, causal, `DriveWF`, `SingleDriver`), every required port is an unresolved declaration of the stated interface, every provided port is declared with it, parameters are unresolved, data-typed and clock-free.

Instance k of a component maps local identity n to fresh W k $n = W \cdot (k + 1) + n$, with decode its inverse; distinct instances never share an identity (`inst_decl_disjoint`). The encoding is a device — any injective allocator would do. A **binding** realizes a destination port of one instance from a source — a port of another instance or a closed constant — with an optional transport: none for a direct reference in the same or an agnostic domain, some *init* for sync from the source’s domain. A **system** is a width, a list of instances, a list of bindings, the shared concept environment and the external outputs. **Flattening** is the union of the renamed instances followed by the bindings applied as §4 realization steps: the destination port is realized as `declRef src` or `sync c init (declRef src)`. The result is a design, consumed by every existing judgment unchanged.

Theorem 25 (Flattening is well formed; `binding_satisfies`, `flatten_WF`, `flatten_causal`, `flatten_wellClocked`, `flatten_singleDriver`, `open_port_stays_open`). Under `ComposeWF` — every instance realizes its interface; every binding is well formed (types agree; a direct binding’s source is in the destination’s domain or agnostic; a transported binding’s source has a domain); external outputs are driven by at most one instance — and with evidence that is monotone, equivariant and port-sound (a discharged commitment survives when a port copy is realized by a reference to a declaration of the same interface), the flattening is globally well formed, well clocked, single-driver, causal when the inter-instance graph is acyclic, and its open ports remain open.

Theorem 26 (Modular semantics, restricted; `eval_flat_to_inst`, `eval_inst_to_flat`, `modular_iff_flat`). For wiring designs with closure-free inputs and direct bindings, in one domain, the value of a declaration in an instance evaluated alone with a consistent modular input equals its value in the flattened system.

The restriction is exact and recorded: transported bindings under `MEv` need a domain-indexed input for the transported port, and higher-order bodies are not covered — the same obstacle in both directions. Substitutability follows the usual contravariance: B may replace A when every port A provides, B provides at the same type

and clock, and every port B requires, A required; replacing an instance by a refining component preserves `ComposeWF` (`substitute_composeWF`).

The counterexamples that fixed the design (`Experiments/BehaviorAlternatives.lean`): a name-based identity collides on double instantiation; a shared clock captured inside a template cannot be re-bound; a binding across domains without transport is rejected by `Clocked`; two instances driving one external output violate `SingleDriver`.

10.3 Groups are the identity, and extraction is a system

A *group* — a designer’s selection of several declarations — is authoring metadata: a group identity and a member list beside the design. Every group operation acts on the list and leaves the design untouched, so every kernel judgment of the design is the *same proposition* before and after, each proved by reflexivity (`group_is_identity_on_design`). A group’s boundary is a projection over a finite enumeration: the non-members some member depends on (`crossIn`), the members some non-member depends on (`crossOut`); the aggregate socket a collapsed group shows is these lists, none of which is a declaration, and $a \in \text{crossIn}$ says *some* member depends on a and nothing about the others (`socket_no_fanout`). Two encodings of a socket as a declaration were refuted: as a declaration every member reads, a member acquires an instantaneous dependency it never had; as a tuple-returning declaration, the consumer of one member comes to depend on the inputs of all of them. The kernel has no tuples for boundaries, and this is a reason not to add them for that purpose.

Packaging a group as a component — the one semantic step in the hierarchy declaration $\rightarrow \text{group} \rightarrow \text{component} \rightarrow \text{system}$ — builds two *restrictions* of the design, the component with an unresolved copy of each crossing-in declaration as a required port and the residual with an unresolved copy of each crossing-out member, and forms a two-instance system with one direct binding per crossing. No body is translated or copied across the boundary. Both templates realize their inferred interfaces (`restrict_realizes`, needing evidence that depends only on the interfaces of the referenced declarations, `Evidence.InterfaceLocal`); the system is a well-formed composition and its flattening a well-formed design (`system_composeWF`, `flat_WF`); causality needed its own argument, since a group with both inputs and outputs is never inter-instance-acyclic, yet the flattened instantaneous graph is the original with every crossing edge subdivided through a port copy, and doubling the original rank witnesses it (`flat_causat`); private members are unobservable from the residual (`private_unobservable`); and for wiring designs an original declaration and its home copy evaluate to the same value at every tick (`orig_iff_flat`).

11 Mechanization

The development is 68 Lean 4 modules (Lean 4.33.1, no dependencies beyond core): 11 in `Core` (the calculus of §3–7 and §9), 12 in `Behavior` (§10), 19 in `Surface` (the definitional library, polymorphism, units, buffering and the boundary constructions above the kernel), 2 in `Validation` (hardware and capacity, outside this paper), and 24 experiment modules holding alternatives, counterexamples and executed examples; about 26 500 lines and 1 545 theorem declarations. It builds with no sorry. The axioms are propositional extensionality and quotient soundness, the latter only through function extensionality and the choice-free rational quotient used by the unit laws; classical choice is absent, and the whole development was re-audited for it at every phase.

Three proof-engineering choices carried the metatheory.

One inductive relation. `Ev` and `MEv` are ordinary inductive relations with no mutual recursion and no fixpoint, because the recursor’s rule unrolls through the environment (§6.1). Every induction on evaluation — determinism, provenance, closure-freeness, environment irrelevance, unfolding — extends by one case when a construct is added; the transport primitive and the recursor entered this way, and every earlier theorem was re-established without a change of statement.

One logical relation, generic in application. $\mathcal{R}$ is parameterized by an application relation so that the single- and multi-domain semantics share it; at data types it is independent of that parameter (`Red_data`), which is the fact that lets a value cross a tick. The lexicographic induction on (tick, rank, derivation) is written once, in `fundamental`, and once more in `mfundamental` with `prevAct` in place of the predecessor.

Counterexamples as theorems, traces by a proved interpreter. Every rejected alternative is a theorem whose content is a rejection, stated on a concrete design and discharged by `decide` or by running `evalF/mevalF` (proved sound for the relations) inside the checker. There is no test suite beside the proofs; the executed examples are part of the same `lake build`. Several results are recorded as trivial by definition — the typing half of client stability, the well-formedness of a refinement target — and are reported as such rather than presented as content.

Extraction to an implementation is not part of the development. The production toolchain implements the calculus in Rust and is tested differentially against the traces the interpreter computes; that correspondence is a tested claim, not a theorem, and is recorded as such in the monograph.

12 Related work

Synchronous languages. λ_{BDL} 's time model is that of Lustre [5] and Esterel [17]: a global logical tick, streams as the meaning of declarations, memory as `pre` with an explicit initial value, causality as acyclicity of instantaneous dependencies. Two departures are deliberate. Clocks are nominal and authored, not inferred [10], [11], [18], because the information that would let a clock be inferred arrives at realization binding, after the design has been reasoned about; and a cross-clock read is a single primitive with a strictly-before rule and an explicit initial value, so that no simultaneously active domains ever see each other and no `when/merge` calculus of sub-sampling is needed — sub-domains evaluated in one instant are the same domain here. The window buffer of §7.5 plays the role that sampling operators play in Lucid Synchrone. Vélus [19] verifies a Lustre compiler; the present development verifies a calculus and its metatheory, and leaves the compiler to differential testing. Zélus [20] and state-machine extensions [21] address continuous time and modes, neither of which the calculus has.

Functional reactive programming. FRP [22], [23], [24] makes signals first-class values. The calculus has no signal type: every declaration is a stream by interpretation, and a signal type would reject nothing (§6.7). Typed FRP with modal or temporal types [25], [26], [27] uses the type to control what may be remembered; here that control is the data restriction on delay and the empty-context restriction, both forced by the totality proof, and the guarantee is totality rather than the absence of space leaks.

Logical relations. The totality proof is a step-indexed logical relation [6], [7] in which the index is the tick and a rank on declarations is a second index; the arrow clause is tick-indexed and the data clauses are not, which is what makes memory sound. Kripke-style monotonicity conditions on a world are standard in such proofs; the evidence-monotonicity condition of §4.3 is the same shape imposed on a *validation layer* rather than on a store.

Modules, abstract types and nominal types. Persistent declarations are the interface/implementation separation of module signatures [2], [3] with a growable commitment set whose growth is a first-class operation on a declared-but-undefined name. The grant is the private constructor of an abstract type [4], [14] with the capability derived from the signature. Nominal type identity is the ordinary mechanism of a nominal type system [28]; the contribution is the pair of erasure and provenance theorems and the refuted alternatives, not the mechanism.

Units of measure. Kennedy's dimension types [8], [9] put dimension polymorphism in the type system. The calculus keeps dimensions monomorphic in operator types and provides dimension polymorphism only in surface families instantiated by matching (§8.4), with units entirely outside the kernel.

Typed holes and live programming. Hazelnut [29], [30] gives a semantics to programs with holes and edits. An unresolved declaration is not a hole position in a term but a declaration whose realization is absent, referred to by identity and typed by its interface; the refinement order plays the role of the edit action calculus, restricted to the operations under which clients are stable.

Effects. The single-driver discipline is not an effect system [16]; §9 records that effect rows and action values, in the toy forms tried, add no rejection the drive edge lacks. Statecharts [31] and model-based design tools supply modes and hierarchy that the calculus encodes as ordinary declarations.

Expressiveness. The negative results of §7.5 and §8 are statements about what a construct can and cannot express relative to the kernel — in Felleisen's sense [32] of whether a construct is definable by a local translation — with

the difference that each is a mechanized theorem about a specific candidate rather than a general expressiveness result.

13 Limitations and open problems

The paper’s claims are bounded by the following, each recorded in the development.

- *Causality is conservative for lambda-guarded cycles.* $A := \lambda x. A \ x$ is rejected by Causal although `declRef A` evaluates to a closure; Theorem 11 covers strict cycles only. A finer criterion that admits productive higher-order cycles has not been formulated.
- *Modular semantics is proved for a fragment.* Theorem 26 holds for single-domain wiring designs with direct or constant bindings. Transported bindings under MEv need a domain-indexed modular input; higher-order bodies need a relation between closures across the two evaluations. Both are open.
- *Evidence is abstract.* The kernel imposes monotonicity, equivariance and port-soundness on the validation layer’s evidence and proves nothing about a concrete discharge mechanism; a compositional evidence model that discharges these once is future work.
- *No sums.* Enumerations with payloads are encoded; a kernel sum would be one eliminator term former, and its absence is a decision to stop where the executed cases stopped.
- *Magnitudes are naturals.* The executable kernel’s quantities are natural numbers, so its unit registry is exact only for integer scales; the unit laws are proved over an abstract scalar domain instantiated symbolically, and floating-point implementations are held to tolerated versions above the kernel.
- *No minimality theorem.* “Minimal” means minimal among the formalized candidates; several alternatives (flow-sensitive semantic analyses, structural typing with a separate role judgment) were argued against rather than refuted.
- *Nothing about designers.* The calculus was shaped by requirements from a design workflow; whether it serves designers is an empirical question no theorem addresses.

14 Conclusion

λ_{BDL} is a small calculus whose content is in its environments and the disciplines they impose rather than in its terms: a declaration environment read by typing through the type view and by evaluation through the realization view, with a refinement order under which clients are stable; a concept environment read through write-once bindings, with a grant derived from the signature under which construction is authorized; a clock environment read by a judgment that typing never sees; and an output environment read by a drive discipline with a single driver. One temporal primitive gives memory and transport, and a tick-indexed logical relation gives totality on causal designs in one domain and in many. The negative results — a scheduler made observable by same-tick visibility, an evidence relation destroyed by a valid realization, a hidden concept crossing admitted by unrestricted construction, three physical outputs from one design under hidden arbitration, a lossy summary for every bounded buffer — are mechanized alongside the positive ones and are, we think, as much a part of the calculus’s specification as the rules.

Appendix A — Theorem index

Every name is a Lean declaration in KCN-judu/BDL_FV; the file is given per group, and names are unqualified where the namespace is BDL.

paper	Lean name	file
Thm 1	<code>infer_sound, infer_complete, HasType.unique</code>	<code>Core/Typing</code>
Thm 2	<code>HasType.mono_env, HasType.mono_concept, HasType.mono_grant</code>	<code>Core/Typing</code>
§4.1	<code>InterfaceRefines_iff_semantic, naive_breaks_wellformedness</code>	<code>Core/Satisfaction, Experiments/DeclCounterexamples</code>
Thm 3	<code>DeclRefinesStar_iff, EnvRefines_update</code>	<code>Core/Satisfaction, Core/Decl</code>
Thm 4	<code>local_refinement_preserves_global_typing</code>	<code>Core/Env</code>

paper	Lean name	file
Thm 5	local_refinement_preserves_global_wf, local_lifecycle_preserves_global_wf	Core/Env
Prop 6	badEv_not_mono, probe6_breaks	Experiments/DeclCounterexamples
Thm 7	Unfolds.det, Unfolds.exists_of_acyclic, Unfolds.not_of_cyclic, Unfolds.refFree_of_fullyRealized	Core/Dependency
Thm 8	HasType.constructs_granted, hidden_crossing_rejected_under_grant	Core/Typing, Experiments/RepresentationBindingAlternatives
Thm 9	HasType.erase, erase_not_injective, baseline_is_erased_modelA	Experiments/SemanticTypeAlternatives
§5.3	dimension_mismatch_rejected, counterexampleB_baseline_accepts_length_plus_time, same_dimension_does_not_imply_same_semantic_identity	Experiments/DimensionAlternatives
Thm 10	Ev.det, evalF_sound	Core/Reactive
Thm 11	Ev.not_of_strictCyclic, Causal_iff_acyclic_of_delayFree	Core/Reactive, Core/Dependency
Thm 12	fundamental, Red_data, Red_prim	Core/Reactive
Thm 13	reactive_total, Ev.red	Core/Reactive
§6.4	arrow_not_delayable, delay_not_under_binder, sync_not_under_binder, first_tick_undefined_without_init	Experiments/PolyAlternatives, Surface/UnitDomain, Experiments/ReactiveAlternatives
Thm 14	Ev.tag_provenance, temporal_state_preserves_semantic_identity	Core/Reactive
Thm 15	unfolds_preserves_eval, Ev.pure, Ev.env_irrelevant	Core/Reactive
Thm 16	delay_is_sync_own, clocked_delay_iff_sync_own	Core/Clock
Thm 17	single_domain_embedding	Core/Clock
Thm 18	MEv.det, mevalF_sound	Core/Clock
Thm 19	mfundamental, multi_domain_total, MEv.tag_provenance	Core/Clock
Thm 20	scheduling_order_observable, equal_rate_not_same_domain, clocked_type_forces_polymorphism	Experiments/ClockAlternatives
Thm 21	buffer_window_correspondence, buffer_from_log_and_cursor, bounded_summary_not_lossless, lossless_iff_injective	Surface/Buffer, Core/Clock, Experiments/BufferAlternatives
Thm 22	fold_total, mfold_total, fold_spec	Core/Reactive, Core/Clock, Surface/Stdlib
§8.2–8.3	church_fst_rank, church_pair_prenex_one_projection, Value.beq_iff, Cap.eq_iff_data, lt_only_on_quantities	Experiments/PolyAlternatives, Surface/Generic, Surface/Poly, Experiments/EquationExamples
§8.4–8.5	matchTy_sound, matchTy_complete, Scheme.instantiate_sound, generic_preserves_identity, lib_expansion	Surface/Poly, Surface/Generic, Surface/Stdlib
Thm 23	single_driver_output_deterministic, multiple_direct_drivers_rejected, hidden_arbitration_observable, first_output_binding_is_monotone	Core/Output, Experiments/OutputAlternatives
§9	driver_is_unit_domain, unit_codomain_collapse, consumers_indistinguishable, eval_independent_of_drives	Surface/UnitDomain
Thm 24	HasType.rename, Satisfies.rename, Clocked.rename	Behavior/Rename
Thm 25	inst_decl_disjoint, binding_satisfies, flatten_WF, flatten_causal, flatten_wellClocked, flatten_singleDriver, open_port_stays_open	Behavior/Instantiate, Behavior/Preservation
Thm 26	eval_flat_to_inst, eval_inst_to_flat, modular_iff_flat, substitute_composeWF	Behavior/Semantics, Behavior/Substitution
§10.3	group_is_identity_on_design, socket_no_fanout, restrict_realizes, system_composeWF, flat_WF, flat_causal, private_unobservable, orig_iff_flat	Behavior/Group, Behavior/Boundary, Behavior/Extract, Behavior/ExtractPreservation

References

- [1] L. de Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Automated Deduction – CADE 28*, in Lecture Notes in Computer Science, vol. 12699. Springer, 2021, pp. 625–635. doi: [10.1007/978-3-030-79876-5_37](https://doi.org/10.1007/978-3-030-79876-5_37).
- [2] X. Leroy, “Manifest Types, Modules, and Separate Compilation,” in *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 1994, pp. 109–122. doi: [10.1145/174675.176926](https://doi.org/10.1145/174675.176926).
- [3] R. Harper and M. Lillibridge, “A Type-Theoretic Approach to Higher-Order Modules with Sharing,” in *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 1994, pp. 123–137. doi: [10.1145/174675.176927](https://doi.org/10.1145/174675.176927).

- [4] J. C. Mitchell and G. D. Plotkin, “Abstract Types Have Existential Type,” *ACM Transactions on Programming Languages and Systems*, vol. 10, no. 3, pp. 470–502, 1988, doi: [10.1145/44501.45065](https://doi.org/10.1145/44501.45065).
- [5] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud, “The Synchronous Data Flow Programming Language LUSTRE,” *Proceedings of the IEEE*, vol. 79, no. 9, pp. 1305–1320, 1991, doi: [10.1109/5.97300](https://doi.org/10.1109/5.97300).
- [6] A. W. Appel and D. McAllester, “An Indexed Model of Recursive Types for Foundational Proof-Carrying Code,” *ACM Transactions on Programming Languages and Systems*, vol. 23, no. 5, pp. 657–683, 2001, doi: [10.1145/504709.504712](https://doi.org/10.1145/504709.504712).
- [7] A. Ahmed, “Step-Indexed Syntactic Logical Relations for Recursive and Quantified Types,” in *Programming Languages and Systems, 15th European Symposium on Programming*, in Lecture Notes in Computer Science, vol. 3924. Springer, 2006, pp. 69–83. doi: [10.1007/11693024_6](https://doi.org/10.1007/11693024_6).
- [8] A. Kennedy, “Relational Parametricity and Units of Measure,” in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 1997, pp. 442–455. doi: [10.1145/263699.263761](https://doi.org/10.1145/263699.263761).
- [9] A. Kennedy, “Types for Units-of-Measure: Theory and Practice,” in *Central European Functional Programming School*, in Lecture Notes in Computer Science, vol. 6299. Springer, 2010, pp. 268–305. doi: [10.1007/978-3-642-17685-2_8](https://doi.org/10.1007/978-3-642-17685-2_8).
- [10] J.-L. Colaço and M. Pouzet, “Clocks as First Class Abstract Types,” in *Proceedings of the Third ACM International Conference on Embedded Software*, ACM, 2003.
- [11] D. Biernacki, J.-L. Colaço, G. Hamon, and M. Pouzet, “Clock-Directed Modular Code Generation for Synchronous Data-Flow Languages,” in *Proceedings of the 2008 ACM SIGPLAN-SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems*, ACM, 2008, pp. 121–130. doi: [10.1145/1375657.1375674](https://doi.org/10.1145/1375657.1375674).
- [12] L. Damas and R. Milner, “Principal Type-Schemes for Functional Programs,” in *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 1982, pp. 207–212. doi: [10.1145/582153.582176](https://doi.org/10.1145/582153.582176).
- [13] B. C. Pierce and D. N. Turner, “Local Type Inference,” *ACM Transactions on Programming Languages and Systems*, vol. 22, no. 1, pp. 1–44, 2000, doi: [10.1145/345099.345100](https://doi.org/10.1145/345099.345100).
- [14] J. C. Reynolds, “Types, Abstraction and Parametric Polymorphism,” in *Information Processing 83, Proceedings of the IFIP 9th World Computer Congress*, North-Holland, 1983, pp. 513–523.
- [15] P. Wadler, “Theorems for Free!,” in *Proceedings of the Fourth International Conference on Functional Programming Languages and Computer Architecture*, ACM, 1989, pp. 347–359. doi: [10.1145/99370.99404](https://doi.org/10.1145/99370.99404).
- [16] G. D. Plotkin and M. Pretnar, “Handling Algebraic Effects,” *Logical Methods in Computer Science*, vol. 9, no. 4, 2013, doi: [10.2168/LMCS-9\(4:23\)2013](https://doi.org/10.2168/LMCS-9(4:23)2013).
- [17] G. Berry and G. Gonthier, “The Esterel Synchronous Programming Language: Design, Semantics, Implementation,” *Science of Computer Programming*, vol. 19, no. 2, pp. 87–152, 1992, doi: [10.1016/0167-6423\(92\)90005-V](https://doi.org/10.1016/0167-6423(92)90005-V).
- [18] P. Caspi and M. Pouzet, “Synchronous Kahn Networks,” in *Proceedings of the First ACM SIGPLAN International Conference on Functional Programming*, ACM, 1996, pp. 226–238. doi: [10.1145/232627.232651](https://doi.org/10.1145/232627.232651).
- [19] T. Bourke, L. Brun, P.-É. Dagand, X. Leroy, M. Pouzet, and L. Rieg, “A Formally Verified Compiler for Lustre,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ACM, 2017, pp. 586–601. doi: [10.1145/3062341.3062358](https://doi.org/10.1145/3062341.3062358).
- [20] T. Bourke and M. Pouzet, “Zélus: A Synchronous Language with ODEs,” in *Proceedings of the 16th International Conference on Hybrid Systems: Computation and Control*, ACM, 2013, pp. 113–118. doi: [10.1145/2461328.2461348](https://doi.org/10.1145/2461328.2461348).

- [21] J.-L. Colaço, B. Pagano, and M. Pouzet, “A Conservative Extension of Synchronous Data-flow with State Machines,” in *Proceedings of the 5th ACM International Conference on Embedded Software*, ACM, 2005. doi: [10.1145/1086228.1086261](https://doi.org/10.1145/1086228.1086261).
- [22] C. Elliott and P. Hudak, “Functional Reactive Animation,” in *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming*, ACM, 1997, pp. 263–273. doi: [10.1145/258948.258973](https://doi.org/10.1145/258948.258973).
- [23] H. Nilsson, A. Courtney, and J. Peterson, “Functional Reactive Programming, Continued,” in *Proceedings of the 2002 ACM SIGPLAN Workshop on Haskell*, ACM, 2002, pp. 51–64. doi: [10.1145/581690.581695](https://doi.org/10.1145/581690.581695).
- [24] G. H. Cooper and S. Krishnamurthi, “Embedding Dynamic Dataflow in a Call-by-Value Language,” in *Programming Languages and Systems: 15th European Symposium on Programming*, Springer, 2006, pp. 294–308.
- [25] N. R. Krishnaswami, “Higher-Order Functional Reactive Programming without Spacetime Leaks,” in *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming*, ACM, 2013, pp. 221–232. doi: [10.1145/2500365.2500588](https://doi.org/10.1145/2500365.2500588).
- [26] A. Jeffrey, “LTL Types FRP: Linear-Time Temporal Logic Propositions as Types, Proofs as Functional Reactive Programs,” in *Proceedings of the Sixth Workshop on Programming Languages meets Program Verification*, ACM, 2012, pp. 49–60. doi: [10.1145/2103776.2103783](https://doi.org/10.1145/2103776.2103783).
- [27] A. Cave, F. Ferreira, P. Panangaden, and B. Pientka, “Fair Reactive Programming,” in *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 2014, pp. 361–372. doi: [10.1145/2535838.2535881](https://doi.org/10.1145/2535838.2535881).
- [28] B. C. Pierce, *Types and Programming Languages*. MIT Press, 2002.
- [29] C. Omar, I. Voysey, M. Hilton, J. Aldrich, and M. A. Hammer, “Hazelnut: A Bidirectionally Typed Structure Editor Calculus,” in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, ACM, 2017, pp. 86–99. doi: [10.1145/3009837.3009900](https://doi.org/10.1145/3009837.3009900).
- [30] C. Omar, I. Voysey, R. Chugh, and M. A. Hammer, “Live Functional Programming with Typed Holes,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, p. 14:1–14:32, 2019, doi: [10.1145/3290327](https://doi.org/10.1145/3290327).
- [31] D. Harel, “Statecharts: A Visual Formalism for Complex Systems,” *Science of Computer Programming*, vol. 8, no. 3, pp. 231–274, 1987, doi: [10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9).
- [32] M. Felleisen, “On the Expressive Power of Programming Languages,” in *Programming Languages and Systems, 3rd European Symposium on Programming*, in Lecture Notes in Computer Science, vol. 432. Springer, 1990, pp. 134–151. doi: [10.1007/3-540-52592-0_60](https://doi.org/10.1007/3-540-52592-0_60).